Fastify with Node.js

The Complete Practical Tutorial

Installation, routing, the request lifecycle, validation & serialization, plugins & encapsulation, decorators, hooks, error handling, logging, the plugin ecosystem, TypeScript, testing, performance, deployment, 14 tested real-world examples and a complete production-style REST API.

Verified with Fastify 5.12 · Node.js 24 · TypeScript 7

Contents

Part I — Getting Started 4

1. Why Fastify? . . . 4
2. Core Architecture . . . 5
3. Installation . . . 6
4. Your First Server . . . 8

Part II — Core Concepts 9

5. Routing . . . 9
6. Request & Reply . . . 12
7. Validation & Serialization . . . 19
8. Plugins & Encapsulation . . . 24
9. Decorators . . . 25
10. Hooks . . . 29
11. Error Handling . . . 33
12. Content-Type Parsers . . . 37
13. Logging with Pino . . . 41

Part III — The Plugin Ecosystem 42

14. Configuration with @fastify/env . . . 42
15. Databases . . . 42
16. Authentication . . . 46
17. Security Plugins . . . 51
18. OpenAPI / Swagger Docs . . . 54
19. Static Files, Uploads & WebSockets . . . 54

Part IV — TypeScript, Testing & Production 60

20. TypeScript in Depth . . . 60
21. Testing . . . 61
22. Project Structure & Autoload . . . 62
23. Performance . . . 63
24. Deployment . . . 63

Part V — End-to-End Project: Tasks API 67

25. Building a Production-Style REST API . . . 67

Appendices 77

A. Express to Fastify Cheat Sheet . . . 77

B. Official Plugins Reference 77
C. Troubleshooting 78

Part I — Getting Started

1. Why Fastify?

Fastify is a fast, low-overhead web framework for Node.js. It is built around three ideas that set it apart from Express:

- **Schema-first**: routes declare JSON Schemas for input and output. Input is validated with Ajv; output is serialized with compiled `fast-json-stringify` functions — both faster and safer than hand-written checks and `JSON.stringify`.
- **Plugins with encapsulation**: everything (routes, hooks, decorators, DB connections) is a plugin. Each plugin gets its own isolated context, so features can't accidentally leak into each other.
- **A defined request lifecycle** with typed hooks at every stage, instead of an untyped middleware chain.

Topic	Express	Fastify
Async handlers	Errors in async code need wrappers (v4)	Native: return a value or throw
Validation	Bring your own (Joi, zod, express-validator)	Built-in JSON Schema (Ajv)
Serialization	`JSON.stringify`	Compiled per-route serializer (2-3x faster)
Logging	Bring your own	Pino built-in, request-scoped `request.log`
Structure	Global middleware stack	Encapsulated plugin tree
TypeScript	Community types	First-class types, type providers
Testing	supertest	Built-in `app.inject()` (no network)

1.1 Real-world examples in this guide

Every major topic is shown twice: first the concept, then a **real-world example** that runs and is covered by tests. All examples live in `fastify-examples/` next to this PDF (54 tests, all passing).

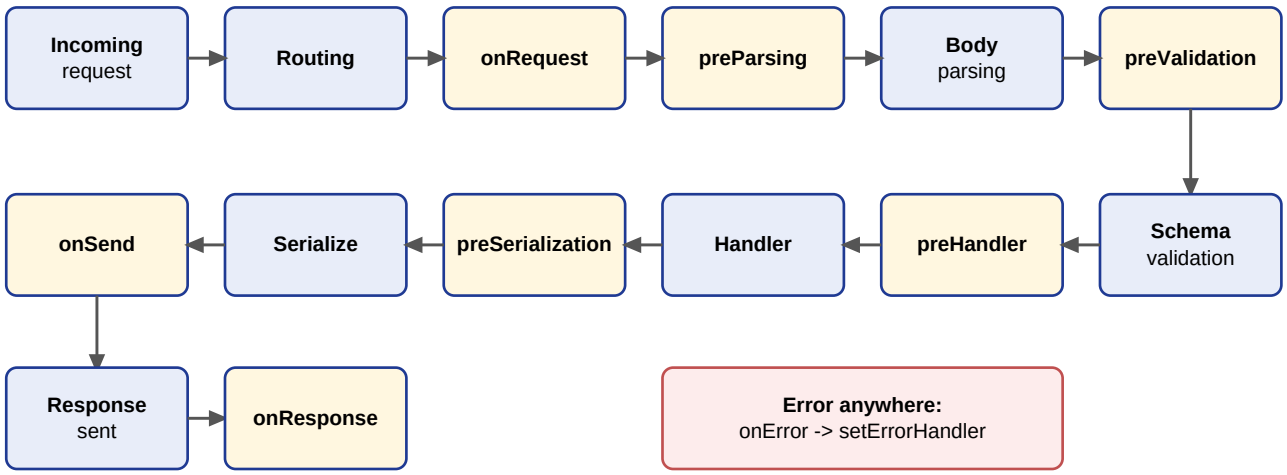
Scenario	Topics	Section
E-commerce catalog	Regex params, wildcards, versioning, prefixes, redirects	5.5
CSV exports, cacheable data	Streams, headers, ETag/304, HEAD	6.4
Live order tracking	Server-Sent Events, hijack, cleanup	6.5
User sign-up	TypeBox validation, field errors, response whitelist, OpenAPI	7.6
Multi-tenant SaaS	fp() vs encapsulation, request decorators	9.1
Production observability	Hooks, request IDs, audit, redaction, maintenance	10.4
Bank transfers	Domain errors, RFC 9457 problem details	11.5
Payment webhooks	Raw-body parser, HMAC signatures, replay + dedupe	12.1
Inventory reservations	Database plugin, transactions, SQL safety	15.3
Login and roles	JWT, refresh rotation, RBAC, API keys	16.4
Hardening a public API	Helmet, CORS, brute-force limits, body limits	17.1
Profile picture uploads	Streaming multipart, magic bytes, limits	19.4

Scenario	Topics	Section
Support chat	WebSockets, auth on upgrade, rooms	19.5
Zero-downtime deploys	Graceful shutdown, readiness, keep-alive	24.4

2. Core Architecture

2.1 The request lifecycle

Every request flows through the same fixed stages. You attach behaviour with **hooks** rather than ordering middleware. Knowing this diagram is the key to understanding Fastify.

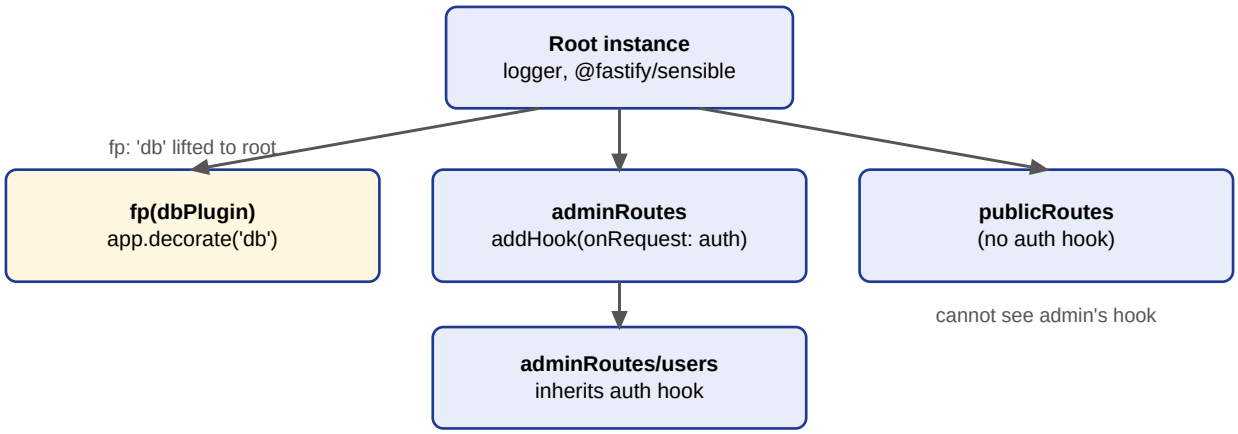


Yellow = hooks you can attach. Blue = Fastify internals.

1. **Routing** — find-my-way (radix tree) matches method + URL. No match → 404 handler.
2. **onRequest** — body not parsed yet. Ideal for auth, rate checks, request IDs.
3. **preParsing** — can transform the raw body stream (e.g. decompress).
4. **Parsing** — content-type parser turns the body into `request.body`.
5. **preValidation** → **Validation** — body, querystring, params, headers checked against schemas. Failure → 400.
6. **preHandler** — last stop before your code (authorization, loading entities).
7. **Handler** — your route function returns data or calls `reply.send()`.
8. **preSerialization** → **Serialization** — object turned into JSON using the response schema for that status code.
9. **onSend** — last chance to change the payload/headers.
10. **onResponse** — after the response is sent. Metrics, cleanup.

2.2 Plugins and encapsulation

A Fastify app is a **tree of plugins**. Each `register()` call creates a child context that inherits from its parent but whose own hooks, decorators and routes are invisible to siblings and parents. Wrapping a plugin with `fastify-plugin` (fp) opts out of encapsulation, so what it adds is shared with the parent — used for shared utilities like DB connections.



Plugins are loaded **asynchronously and in order** by a boot loader (avvio). The server starts listening only after every plugin is ready.

3. Installation

3.1 Prerequisites

- **Node.js 20 or newer** (Fastify v5 requirement). Check with `node -v`.
- npm, pnpm or yarn.
- Optional: `curl` or a REST client, Docker for deployment.

3.1a Tested with

The complete Part V project, its tests and every behaviour described in this guide were run against these versions (September 2026). The runnable projects are `fastify-tasks-api/` (Part V) and `fastify-examples/` (14 real-world examples, 54 tests), next to this PDF.

Component	Version	Notes
Node.js	24.15	Fastify 5 needs 20+
`fastify`	5.12.5	
`@fastify/type-provider-typebox`	6.1.0	re-exports `Type` from the `typebox` 1.x package
`@fastify/jwt` / `@fastify/swagger` / `@fastify/swagger-ui`	10.2 / 9.9 / 6.1	
`@fastify/cors` / `@fastify/helmet` / `@fastify/rate-limit`	11.3 / 13.1 / 11.2	
`@fastify/env` / `@fastify/sensible`	7.0 / 6.0	
`fastify-cli`	8.0.2	
TypeScript / tsx	7.0.2 / 4.23	TS 7 = native compiler; run with tsx

3.2 Option A: scaffold with fastify-cli

```
npx fastify-cli generate my-api --lang=ts     # omit --lang for JavaScript
cd my-api
npm install
npm run dev                                   # http://localhost:3000
```

The generated project uses `@fastify/autoload` with `plugins/` and `routes/` folders and a ready-to-run test setup.

3.3 Option B: manual setup (recommended to learn)

```
mkdir fastify-demo && cd fastify-demo
npm init -y
npm install fastify fastify-plugin
npm install -D typescript tsx @types/node pino-pretty
npx tsc --init
```

tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "nodenext",
    "moduleResolution": "nodenext",
    "strict": true,
    "esModuleInterop": true,
    "skipLibCheck": true,
    "sourceMap": true,
    "rootDir": "./src",
    "outDir": "./dist"
  },
  "include": ["src/**/*.ts"]
}
```

package.json scripts

```
"scripts": {
  "dev":   "tsx watch src/server.ts",
  "build": "tsc",
  "start": "node dist/server.js",
  "test":  "node --import tsx --test test/*.test.ts"
}
```

Watch out: TypeScript 7 removed `"moduleResolution": "node"` (use `"nodenext"`) and has no JavaScript API, so `ts-node`/`ts-jest` no longer work. Use `tsx` and `node --test`.

Tip: `tsx` runs TypeScript directly with fast reloads. In production, compile with `tsc` and run plain JavaScript.

3.4 JavaScript (CommonJS and ESM)

```
// CommonJS - server.js
const fastify = require('fastify')({ logger: true });

// ES modules - server.mjs (or "type": "module" in package.json)
import Fastify from 'fastify';
const fastify = Fastify({ logger: true });
```

4. Your First Server

```
// src/server.ts
import Fastify from 'fastify';

const app = Fastify({ logger: true });

app.get('/', async (request, reply) => {
  return { hello: 'world' };            // object -> JSON, status 200
});

const start = async () => {
  try {
    await app.listen({ port: 3000, host: '0.0.0.0' });
  } catch (err) {
    app.log.error(err);
    process.exit(1);
  }
};
start();
```

Run it

```
npm run dev
curl http://localhost:3000/
# {"hello":"world"}
# server log (JSON, pino):
# {"level":30,"reqId":"req-1","req":{"method":"GET","url":"/"},
#  "msg":"incoming request"}
```

4.1 Important server options

Option	Default	Purpose
`logger`	false	true, or a pino config object (level, redact, transport).
`bodyLimit`	1 MiB	Max request body size in bytes.
`trustProxy`	false	Trust X-Forwarded-* headers behind a load balancer.
`connectionTimeout`	0	Socket timeout in ms.
`requestTimeout`	0	Max time to receive the full request (set in prod).
`genReqId`	counter	Custom request-ID generator (e.g. UUIDs).
`requestIdHeader`	false	Reuse an incoming ID header, e.g. 'x-request-id'.
`ajv`	—	Customize the Ajv validator (see Chapter 7).
`http2` / `https`	—	Enable HTTP/2 or TLS directly.

4.2 listen() details

- Always pass an object: `{ port, host }` (v5 removed positional args).
- Use `host: '0.0.0.0'` in Docker/Kubernetes; the default `localhost` is not reachable from outside the container.
- `port: 0` picks a random free port (handy in tests).
- `await app.ready()` loads all plugins without listening (used by tests).

Part II — Core Concepts

5. Routing

5.1 Shorthand and full declarations

```
app.get('/users', listUsers);
app.post('/users', createUser);
app.put('/users/:id', replaceUser);
app.patch('/users/:id', updateUser);
app.delete('/users/:id', deleteUser);
app.head('/ping', ping);
app.all('/any', anyMethod);                  // every method

// Full declaration - everything in one object
app.route({
  method: ['GET', 'HEAD'],
  url: '/articles/:slug',
  schema: { params: { type: 'object', properties: { slug: { type: 'string' } } } },
  preHandler: [loadArticle],                 // route-level hook
  handler: async (request) => request.article,
});
```

5.2 URL parameters, wildcards and query strings

```
// Named params
app.get('/users/:userId/posts/:postId', async (req) => req.params);
// GET /users/7/posts/42 -> { "userId": "7", "postId": "42" }

// Regex-constrained param (only digits)
app.get('/orders/:id(^\\d+)', async (req) => ({ id: req.params.id }));

// Multiple params in one segment
app.get('/range/:from-:to', async (req) => req.params);    // /range/10-20

// Wildcard
app.get('/files/*', async (req) => ({ path: req.params['*'] }));

// Query string: GET /search?q=node&page=2
app.get('/search', async (req) => req.query);              // { q: 'node', page: '2' }
```

Tip: Without a schema, params and query values are strings. With a schema, Fastify **coerces** them to the declared types (`page` becomes the number 2).

5.3 Async handlers vs reply.send()

```
// Preferred: async + return
app.get('/a', async () => ({ ok: true }));

// Set status/headers, then return
app.post('/b', async (request, reply) => {
  reply.code(201).header('Location', '/b/1');
  return { id: 1 };
});

// Callback style (no async) - must call reply.send exactly once
app.get('/c', (request, reply) => { reply.send({ ok: true }); });
```

Watch out: In an `async` handler, either **return** the value or call `reply.send()` and **return reply**. Calling send() and also returning a different value logs a "Reply already sent" warning.

5.4 Route prefixes and constraints

```
// Prefix all routes of a plugin
app.register(async (api) => {
  api.get('/status', async () => ({ ok: true }));           // GET /api/v1/status
}, { prefix: '/api/v1' });

// Version constraint (Accept-Version header, semver)
app.get('/items', { constraints: { version: '2.0.0' } }, async () => 'v2 items');
// curl -H "Accept-Version: 2.x" localhost:3000/items

// Host constraint
app.get('/', { constraints: { host: 'admin.example.com' } }, adminHome);
```

5.5 Real-world example: an e-commerce catalog API

The situation. A shop exposes products by SKU, category pages of any depth, and a price endpoint whose response format changed (v1 cents, v2 money object) while old mobile apps are still in use. The old PHP site's links must keep working.

Why it works well in Fastify:

- Regex parameters reject malformed IDs at the router.
- Version constraints serve two contracts on one URL.
- Prefixes and redirects keep URL design flexible.

```
// src/catalog/app.ts
import Fastify from 'fastify';
import {
  Type,
  TypeBoxTypeProvider,
  FastifyPluginAsyncTypebox,
} from '@fastify/type-provider-typebox';

interface Product {
  sku: string;
  name: string;
  priceCents: number;
  currency: string;
}
```

```
const PRODUCTS = new Map<string, Product>([
  [
    'PHN-10021',
    { sku: 'PHN-10021', name: 'Pixel 10', priceCents: 79900, currency: 'USD' },
  ],
  [
    'LAP-20450',
    { sku: 'LAP-20450', name: 'ThinkPad X1', priceCents: 189900, currency: 'USD' },
  ],
]);

const catalogRoutes: FastifyPluginAsyncTypebox = async (app) => {
  // Regex-constrained parameter: only well-formed SKUs (ABC-12345) match this route
  app.get('/products/:sku(^[A-Z]{3}-\\d{5}$)', async (request, reply) => {  1
    const { sku } = request.params as { sku: string };
    const product = PRODUCTS.get(sku);
    if (!product) return reply.code(404).send({ message: `No product ${sku}` });
    return product;
  });
```

1 Regex parameter: `/products/phone` doesn't even match this route — no handler code needed to reject it.

```
  // Wildcard: /categories/electronics/phones/android -> breadcrumb
  app.get('/categories/*', async (request) => {  2
    const path = (request.params as { '*': string })['*'];
    return { breadcrumb: path.split('/').filter(Boolean) };
  });

  // Same URL, two contracts: clients choose with the Accept-Version header
  const PriceParams = Type.Object({ sku: Type.String() });
  app.get(
    '/prices/:sku',
    {
      constraints: { version: '1.0.0' },  3
      schema: { params: PriceParams },
    },
    async (request) => ({ sku: request.params.sku, price: 79900 }),
  ); // v1: cents
```

2 Wildcard: everything after `/categories/` arrives in `params['*']`.

3 Version constraint: matched against the `Accept-Version` header (semver ranges like `1.x` work).

```
  const v2 = async (sku: string) => ({
    sku,
    price: { amount: '799.00', currency: 'USD' }, // v2: money
  });
  app.get(
    '/prices/:sku',
    {
      constraints: { version: '2.0.0' },
      schema: { params: PriceParams },
    },
    async (request) => v2(request.params.sku),
  );
  // No Accept-Version header: serve the latest contract
  app.get('/prices/:sku', { schema: { params: PriceParams } }, async (request) =>  4
    v2(request.params.sku),
  );
};
```

4 The unversioned twin answers clients that send no header — here with the latest contract.

```
export async function buildCatalogApp() {
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();

  // Everything public lives under /api (one place to change it)
  await app.register(catalogRoutes, { prefix: '/api' });  5

  // Old PHP site links still in Google: permanent redirect, keep SEO value
  app.get(
    '/item.php',
    {
      schema: { querystring: Type.Object({ id: Type.String() }) },
    },
    async (request, reply) =>
      reply.redirect(`/api/products/${encodeURIComponent(request.query.id)}`, 301),  6
  );

  return app;
}
```

5 A prefix applies to every route in the plugin.

6 301 keeps search-engine ranking when URLs change; always encode user input placed in a URL.

What the tests prove (`test/catalog.test.ts`, all passing):

Test	Verified behaviour
a well-formed SKU matches the product route	PHN-10021 → 200; unknown but valid SKU → handler's 404.
a malformed SKU never reaches the handler	`/api/products/phone` → router 404 ("Route ... not found").
the wildcard turns a category path into a breadcrumb	[electronics, phones, android].
Accept-Version selects the contract; no header gets the latest	`1.x` → cents, `2.x` and no header → money object.
legacy URLs are permanently redirected	`/item.php?id=LAP-20450` → 301 to the API URL.

6. Request & Reply

6.1 The request object

Property	Description
`request.params`, `query`, `body`, `headers`	Parsed (and validated/coerced) input.
`request.id`	Request ID (also in every log line).
`request.log`	Child logger bound to this request.
`request.ip` / `ips` / `hostname` / `protocol`	Client info (respects trustProxy).
`request.method` / `url`	HTTP method and raw URL.
`request.routeOptions`	Matched route info: url pattern, method, config, schema.
`request.raw`	Underlying Node `IncomingMessage`.
`request.server`	The Fastify instance for this context.

6.2 The reply object

```
app.get('/demo', async (request, reply) => {
  reply
    .code(200)                                // status
    .header('Cache-Control', 'no-store')      // single header
    .headers({ 'X-A': '1', 'X-B': '2' })      // several
    .type('application/json');                // content-type
  return { ok: true };
});

app.get('/old', async (req, reply) => reply.redirect('/new', 301));

app.get('/text', async (req, reply) => {
  reply.type('text/plain');
  return 'plain text';
});

// Streams and buffers are sent as-is
import { createReadStream } from 'node:fs';
app.get('/log', async (req, reply) => {
  reply.type('text/plain');
  return createReadStream('./app.log');
});

// reply.elapsedTime -> ms since request start; reply.sent -> already sent?
```

6.3 Taking over the response (hijack) - Server-Sent Events

```
app.get('/events', (request, reply) => {
  reply.hijack();                           // Fastify will not touch the response
  reply.raw.writeHead(200, {
    'Content-Type': 'text/event-stream',
    'Cache-Control': 'no-cache',
    Connection: 'keep-alive',
  });
  const timer = setInterval(() => {
    reply.raw.write(`data: ${JSON.stringify({ now: Date.now() })}\n\n`);
  }, 1000);
  request.raw.on('close', () => clearInterval(timer));
});
```

6.4 Real-world example: CSV exports and cacheable product data

The situation. Accounting downloads a month of orders as CSV (hundreds of thousands of rows); the storefront polls product data that rarely changes.

Why it works well in Fastify:

- Return a stream and Fastify pipes it: constant memory for any export size.
- Headers (`Content-Disposition`, `ETag`) are one call each.
- HEAD requests are answered automatically.

```
// src/downloads/app.ts
import Fastify from 'fastify';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import { Readable } from 'node:stream';
import { createHash } from 'node:crypto';

export interface Order {
  id: string;
  date: string;
  customer: string;
  totalCents: number;
}

export interface OrderSource {
  // Async iterator over a big table: rows are never all in memory at once
  orders(from: string, to: string): AsyncIterable<Order>;  1
}
```

1 Rows come from an async iterator (DB cursor): memory stays flat for any export size.

```
const csvCell = (v: string | number) =>
  /[",\n]/.test(String(v)) ? `"${String(v).replace(/"/g, '""')}"` : String(v);  2

export async function buildDownloadsApp(source: OrderSource) {
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();

  // Accounting's monthly export: streamed straight from the database to the client
  app.get(
    '/exports/orders.csv',
    {
      schema: {
        querystring: Type.Object({
          from: Type.String({ format: 'date' }),  3
          to: Type.String({ format: 'date' }),
        }),
      },
    },
    async (request, reply) => {
      const { from, to } = request.query;
      async function* lines() {
        yield 'id,date,customer,total\n';
        for await (const o of source.orders(from, to)) {
          yield [o.id, o.date, o.customer, (o.totalCents / 100).toFixed(2)]
            .map(csvCell)
            .join(',') + '\n';
        }
      }
      reply
        .type('text/csv; charset=utf-8')
        .header(
          'content-disposition',
          `attachment; filename="orders-${from}_${to}.csv"`,  4
        );
      return Readable.from(lines()); // a stream is sent as it is produced  5
    },
  );
```

2	CSV escaping: quotes, commas and newlines inside a value must be quoted (tested with `Smith, "Jo"`).
3	Bad dates are rejected before any database work starts.
4	`Content-Disposition: attachment` makes browsers download instead of display.
5	Return a stream and Fastify pipes it to the socket chunk by chunk.

```
  // Product page data with conditional GET: unchanged = 304, no body
  const products = new Map([
    ['42', { id: '42', name: 'Desk lamp', priceCents: 3900 }],
  ]);
  app.get(
    '/products/:id',
    {
      schema: { params: Type.Object({ id: Type.String() }) },
    },
    async (request, reply) => {
      const product = products.get(request.params.id);
      if (!product) return reply.code(404).send({ message: 'Not found' });
      const hash = createHash('sha1').update(JSON.stringify(product));
      const etag = `"${hash.digest('base64url')}"`;  6
      reply
        .header('etag', etag)
        .header('cache-control', 'private, max-age=0, must-revalidate');
      if (request.headers['if-none-match'] === etag) return reply.code(304).send();  7
      return product;
    },
  );

  return app;
}
```

6	An ETag fingerprints the representation...
7	...so a client that already has it gets 304 with no body: less bandwidth, faster pages.

What the tests prove (`test/downloads.test.ts`, all passing):

Test	Verified behaviour
streams a CSV attachment with correctly escaped cells	1,000 rows streamed; attachment filename; `Smith, "Jo"` escaped.
rejects an invalid date range before touching the database	`from=yesterday` → 400; the source was never called.
conditional GET: a matching ETag gets 304 with no body	304, empty body.
HEAD requests are answered automatically, without a body	200 with the ETag header, no body.

6.5 Real-world example: live order tracking (Server-Sent Events)

The situation. Customers keep the tracking page open while the order moves from PLACED to DELIVERED. Couriers and the warehouse report status changes.

Why it works well in Fastify:

- SSE needs only `reply.hijack()` and a few headers; browsers reconnect by themselves.
- Simpler than WebSockets for one-way server-to-browser updates.
- Cleaning up on disconnect is the part people forget — it's tested.

```ts
// src/order-tracking/app.ts
import Fastify from 'fastify';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import { EventEmitter } from 'node:events';

export const STATUSES = [
  'PLACED',
  'PACKED',
  'SHIPPED',
  'OUT_FOR_DELIVERY',
  'DELIVERED',
];

// Disconnects are only noticed on the next write: the ping interval bounds how  1
// long a closed tab keeps its subscription
export async function buildTrackingApp({ pingEveryMs = 15_000 } = {}) {
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();
  // In production: Redis pub/sub or a message broker, so every instance sees events
  const bus = new EventEmitter();  2
  const latest = new Map<string, string>();
```

1 Measured: with a 15 s ping, a closed tab stayed subscribed 15 s; with 200 ms it was cleaned up in ~240 ms.

2 In-process only. With several instances, publish through Redis/NATS so every instance can notify its own subscribers.

```ts
  // Courier / warehouse systems report progress
  app.post(
    '/orders/:id/status',
    {
      schema: {
        params: Type.Object({ id: Type.String() }),
        body: Type.Object({
          status: Type.Union(STATUSES.map((s) => Type.Literal(s))),  3
        }),
      },
    },
    async (request, reply) => {
      latest.set(request.params.id, request.body.status);
      bus.emit(request.params.id, request.body.status);
      return reply.code(204).send();
    },
  );
```

3 Only known statuses are accepted from the courier systems.

```
  // The customer's tracking page subscribes with EventSource (Server-Sent Events)
  app.get(
    '/orders/:id/events',
    {
      schema: { params: Type.Object({ id: Type.String() }) },
    },
    (request, reply) => {
      const { id } = request.params;
      reply.hijack(); // we write the response ourselves  4
      reply.raw.writeHead(200, {
        'content-type': 'text/event-stream',  5
        'cache-control': 'no-cache',
        connection: 'keep-alive',
      });
      const send = (status: string) => {
        reply.raw.write(
          `event: status\ndata: ${JSON.stringify({ id, status })}\n\n`,
        );
        if (status === 'DELIVERED') reply.raw.end(); // nothing more will happen
      };
      if (latest.has(id)) send(latest.get(id)!); // current state first  6
      bus.on(id, send);
      const ping = () => reply.raw.write(': ping\n\n'); // SSE comment line
      const keepAlive = setInterval(ping, pingEveryMs);
      reply.raw.on('close', () => {  7
        // tab closed / delivered
        clearInterval(keepAlive);
        bus.off(id, send);
      });
    },
  );
```

4	`hijack()`: Fastify stops managing this response; we own `reply.raw`.
5	The SSE media type; the browser's `EventSource` reconnects automatically.
6	Late subscribers first get the current state, then live updates.
7	Always unsubscribe on close, or every visit leaks a listener.

```
  app.decorate('subscribers', (id: string) => bus.listenerCount(id));
  return app;
}

declare module 'fastify' {
  interface FastifyInstance {
    subscribers(orderId: string): number;
  }
}
```

```
// test/order-tracking.test.ts
// SSE needs a real socket: this test listens on a random port and uses fetch()
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { setTimeout as delay } from 'node:timers/promises';
import { buildTrackingApp } from '../src/order-tracking/app';

let app: Awaited<ReturnType<typeof buildTrackingApp>>;
let base: string;

before(async () => {
  app = await buildTrackingApp({ pingEveryMs: 200 });
  base = await app.listen({ port: 0, host: '127.0.0.1' });  1
});
after(() => app.close());
```

1	Streaming needs a real socket: listen on a random free port.

```
const setStatus = (id: string, status: string) =>
  fetch(`${base}/orders/${id}/status`, {
    method: 'POST',
    headers: { 'content-type': 'application/json' },
    body: JSON.stringify({ status }),
  });

async function readEvents(res: Response, until: (events: string[]) => boolean) {
  const reader = res.body!.pipeThrough(new TextDecoderStream()).getReader();  2
  const events: string[] = [];
  let buffer = '';
  while (!until(events)) {
    const { value, done } = await reader.read();
    if (done) break;
    buffer += value;
    for (const m of buffer.matchAll(/data: (.*)\n\n/g))
      events.push(JSON.parse(m[1]).status);
    buffer = buffer.replace(/[\s\S]*\n\n/, '');
  }
  return { events, reader };
}
```

2 Read the event stream incrementally with `fetch`, like a browser.

```
test('the tracking page receives every status change as it happens', async () => {
  await setStatus('o1', 'PLACED');
  const res = await fetch(`${base}/orders/o1/events`);
  assert.equal(res.headers.get('content-type'), 'text/event-stream');

  const producer = (async () => {
    for (const s of ['PACKED', 'SHIPPED', 'OUT_FOR_DELIVERY', 'DELIVERED']) {
      await delay(50);
      await setStatus('o1', s);
    }
  })();
  const { events } = await readEvents(res, (e) => e.at(-1) === 'DELIVERED');
  await producer;
  assert.deepEqual(events, [
    'PLACED',
    'PACKED',
    'SHIPPED',
    'OUT_FOR_DELIVERY',
    'DELIVERED',
  ]);
});
```

```
test('closing the page unsubscribes (no leaked listeners)', async () => {
  const controller = new AbortController();
  const res = await fetch(`${base}/orders/o2/events`, {
    signal: controller.signal,
  });
  await setStatus('o2', 'PACKED');
  await readEvents(res, (e) => e.length === 1);
  assert.equal(app.subscribers('o2'), 1);
  controller.abort(); // user closes the tab  3
  for (let i = 0; i < 50 && app.subscribers('o2') > 0; i++) await delay(20);
  assert.equal(app.subscribers('o2'), 0);
});
```

3 Simulate the user leaving; then assert the listener is gone.

What the tests prove (`test/order-tracking.test.ts`, all passing):

Test	Verified behaviour
the tracking page receives every status change as it happens	PLACED (current state) then four live updates; the stream ends at DELIVERED.
closing the page unsubscribes (no leaked listeners)	Listener count goes 1 → 0 after the client aborts (~240 ms with a 200 ms ping).

7. Validation & Serialization

Every route may have a `schema` with `body`, `querystring`, `params`, `headers` and `response` keys. Input schemas are compiled with **Ajv**; response schemas with **fast-json-stringify**. Both are compiled once at startup.

7.1 Validating input

```
const createUserSchema = {
  body: {
    type: 'object',
    required: ['email', 'name'],
    additionalProperties: false,
    properties: {
      email: { type: 'string', format: 'email' },
      name:  { type: 'string', minLength: 2, maxLength: 80 },
      age:   { type: 'integer', minimum: 13 },
      role:  { type: 'string', enum: ['user', 'admin'], default: 'user' },
    },
  },
  querystring: {
    type: 'object',
    properties: { notify: { type: 'boolean', default: false } },
  },
  headers: {
    type: 'object',
    properties: { 'x-tenant-id': { type: 'string' } },
    required: ['x-tenant-id'],
  },
} as const;

app.post('/users', { schema: createUserSchema }, async (request, reply) => {
  reply.code(201);
  return { id: 1, ...request.body };   // body.role defaulted to 'user'
});
```

Invalid request -> automatic 400

```
curl -X POST localhost:3000/users -H "Content-Type: application/json" \
     -H "x-tenant-id: t1" -d '{"email":"bad","name":"A"}'
# 400 {"statusCode":400,"code":"FST_ERR_VALIDATION","error":"Bad Request",
#      "message":"body/email must match format \"email\""}
```

Watch out: Fastify v5 requires **full JSON Schemas** for querystring, params and headers (an object with `type: 'object'` and `properties`). The old shorthand `{ q: { type: 'string' } }` is no longer accepted.

7.2 Default Ajv behaviour

Ajv option	Fastify default	Effect
`coerceTypes`	'array'	"42" → 42 for integer fields; single value → array when schema says array.
`useDefaults`	true	Missing properties get their `default`.
`removeAdditional`	true	Extra props removed when `additionalProperties: false` is set.
`allErrors`	false	Stop at the first error (safer against DoS).

```
// Customize (e.g. add formats plugin, change behaviour)
const app = Fastify({
  ajv: {
    customOptions: { removeAdditional: 'all', allErrors: false },
    plugins: [require('ajv-formats')],     // Fastify already includes common formats
  },
});
```

7.3 Response schemas: speed and safety

```
app.get('/users/:id', {
  schema: {
    params: { type: 'object', properties: { id: { type: 'integer' } } },
    response: {
      200: {
        type: 'object',
        properties: { id: { type: 'integer' }, email: { type: 'string' },
                      name: { type: 'string' } },
      },
      404: { $ref: 'errorResponse#' },
    },
  },
}, async (request) => {
  const user = await db.users.find(request.params.id);
  // user contains passwordHash - but it is NOT in the schema, so it is NOT sent
  return user;
});
```

Tip: A response schema is also a security feature: fields not listed (password hashes, internal flags) are silently dropped from the output.

7.4 Shared schemas with $ref

```
app.addSchema({
  $id: 'errorResponse',
  type: 'object',
  properties: {
    statusCode: { type: 'integer' }, error: { type: 'string' },
    message: { type: 'string' },
  },
});
app.addSchema({
  $id: 'user',
  type: 'object',
  properties: { id: { type: 'integer' }, name: { type: 'string' } },
});

app.get('/users', {
  schema: { response: { 200: { type: 'array', items: { $ref: 'user#' } } } },
}, listUsers);
```

7.5 Type providers: one schema, full TypeScript types

```
npm install @fastify/type-provider-typebox
```

```
import Fastify from 'fastify';
import { TypeBoxTypeProvider, Type } from '@fastify/type-provider-typebox';

const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();

const Product = Type.Object({
  id: Type.Integer(),
  name: Type.String({ minLength: 1 }),
  price: Type.Number({ minimum: 0 }),
  tags: Type.Array(Type.String(), { default: [] }),
});

app.post('/products', {
  schema: {
    body: Type.Omit(Product, ['id']),
    response: { 201: Product },
  },
}, async (request, reply) => {
  // request.body is typed: { name: string; price: number; tags: string[] }
  const product = { id: 1, ...request.body };
  reply.code(201);
  return product;                       // checked against Product at compile time
});
```

Alternatives: `@fastify/type-provider-json-schema-to-ts` (plain JSON Schema objects with `as const`) or community Zod providers (`fastify-type-provider-zod`).

7.6 Real-world example: user sign-up

The situation. A sign-up form posts email, password, name, country and a marketing opt-in. The form wants every error at once; the API must never return the password hash; partners need OpenAPI docs.

Why it works well in Fastify:

- One TypeBox schema validates, coerces, documents and types the handler.
- Response schemas whitelist output fields.
- OpenAPI is generated from the same schemas — it can't drift.

```
// src/signup/app.ts
import Fastify, { FastifyError } from 'fastify';
import swagger from '@fastify/swagger';
import { Type, Static, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import { randomUUID, scryptSync, randomBytes } from 'node:crypto';

const SignupBody = Type.Object(
  {
    email: Type.String({ format: 'email', maxLength: 254 }),  1
    password: Type.String({
      minLength: 12,
      maxLength: 128,
      pattern: '^(?=.*[A-Za-z])(?=.*\\d).*$',  2
    }), // letter + digit
    name: Type.String({ minLength: 1, maxLength: 80 }),
    country: Type.Union([
      Type.Literal('IN'),
      Type.Literal('US'),
      Type.Literal('DE'),
    ]),
    // Optional + default: filled in when missing, documented as NOT required
    marketingOptIn: Type.Optional(Type.Boolean({ default: false })),  3
  },
  { additionalProperties: false },  4
);
```

1	Formats (email, date, uri, uuid...) are checked by Ajv.
2	Password policy in the schema: length + at least one letter and one digit.
3	Found while testing: with `default` but without `Optional`, OpenAPI listed the field as **required**.
4	Unknown fields (e.g. `id`, `passwordHash`) are stripped before the handler runs.

```
const User = Type.Object({
  id: Type.String(),
  email: Type.String(),
  name: Type.String(),
  country: Type.String(),
  marketingOptIn: Type.Boolean(),
});
type StoredUser = Static<typeof User> & { passwordHash: string };
const ErrorReply = Type.Object({ message: Type.String() });

export async function buildSignupApp() {
  const app = Fastify({
    // A form wants ALL field errors at once (allErrors costs a little more CPU)
    ajv: { customOptions: { allErrors: true } },  5
  }).withTypeProvider<TypeBoxTypeProvider>();
```

5	Report every invalid field at once (default stops at the first).

```
await app.register(swagger, {
  openapi: { info: { title: 'Accounts', version: '1.0.0' } },
});

const users = new Map<string, StoredUser>();

// Field-level errors the sign-up form can show next to each input
app.setErrorHandler((error: FastifyError, request, reply) => {
  if (error.validation) {
    return reply.code(400).send({
      message: 'Please correct the highlighted fields',
      fields: error.validation.map((v) => ({  6
        field:
          v.instancePath.slice(1) ||
          String((v.params as { missingProperty?: string }).missingProperty),
        problem: v.message,
      })),
    });
  }
  reply.send(error);
});
```

6 Turn Ajv errors into a list the form can show next to each input.

```
app.post(
  '/users',
  {
    schema: { body: SignupBody, response: { 201: User, 409: ErrorReply } },  7
  },
  async (request, reply) => {
    const { password, ...profile } = request.body;
    if (
      [...users.values()].some((u) => u.email === profile.email.toLowerCase())
    ) {
      return reply.code(409).send({ message: 'Email already registered' });
    }
    const salt = randomBytes(16).toString('hex');
    const user: StoredUser = {
      id: randomUUID(),
      ...profile,
      email: profile.email.toLowerCase(),
      marketingOptIn: profile.marketingOptIn ?? false,
      passwordHash: `${salt}:${scryptSync(password, salt, 32).toString('hex')}`,
    };
    users.set(user.id, user);
    reply.code(201);
    return user; // passwordHash is not in the 201 schema: never sent  8
  },
);
```

7 Declare **every** status the route returns. Found by the compiler: without 409 here, `reply.code(409)` was a type error.

8 The response schema is a whitelist: the hash cannot leak, even by accident.

```
  app.get(
    '/users',
    {
      schema: {
        querystring: Type.Object({
          country: Type.Optional(Type.String()),
          optedIn: Type.Optional(Type.Boolean()),
          limit: Type.Integer({ minimum: 1, maximum: 100, default: 20 }),
        }),
        response: { 200: Type.Array(User) },
      },
    },
    async (request) => {
      const { country, optedIn, limit } = request.query; // already typed and coerced  9
      return [...users.values()]
        .filter((u) => !country || u.country === country)
        .filter((u) => optedIn === undefined || u.marketingOptIn === optedIn)
        .slice(0, limit);
    },
  );

  return app;
}
```

9 `?optedIn=true&limit;=5` arrives as a boolean and a number, with a default for limit.

What the tests prove (`test/signup.test.ts`, all passing):

Test	Verified behaviour
valid sign-up: 201, defaults applied, password hash never sent	Email lower-cased, opt-in defaulted to false, no hash or password in the response.
every invalid field is reported at once	email, password, name and country all listed.
unknown properties are stripped: no setting internal fields	`id: 'admin'` in the body is ignored.
query strings are coerced to the schema types	`optedIn=true` filters as a boolean; `limit=500` → 400.
the same schemas generate the OpenAPI document	Required fields and `minLength: 12` appear in the spec.

8. Plugins & Encapsulation

8.1 Writing and registering a plugin

```
// src/routes/users.ts
import type { FastifyPluginAsync } from 'fastify';

interface UsersOptions { maxPageSize: number }

const usersRoutes: FastifyPluginAsync<UsersOptions> = async (app, opts) => {
  app.get('/', async () => ({ pageSize: opts.maxPageSize, users: [] }));
  app.get('/:id', async (req) => ({ id: (req.params as { id: string }).id }));
};
export default usersRoutes;

// src/app.ts
app.register(usersRoutes, { prefix: '/users', maxPageSize: 50 });
```

8.2 Encapsulation in action

```
app.register(async function adminContext(admin) {
  admin.addHook('onRequest', requireAdmin);     // only for routes in this context
  admin.decorate('auditLog', writeAudit);       // only visible here (and children)
  admin.get('/admin/stats', async () => stats());
});

app.register(async function publicContext(pub) {
  pub.get('/public/info', async () => info());  // requireAdmin does NOT run here
  // pub.auditLog -> undefined
});
```

8.3 Sharing with fastify-plugin

```
// src/plugins/db.ts
import fp from 'fastify-plugin';
import { Pool } from 'pg';

declare module 'fastify' {
  interface FastifyInstance { db: Pool }
}

export default fp(async (app, opts: { connectionString: string }) => {
  const pool = new Pool({ connectionString: opts.connectionString });
  await pool.query('SELECT 1');                // fail fast at boot
  app.decorate('db', pool);
  app.addHook('onClose', async () => { await pool.end(); });
}, { name: 'db' });

// app.ts
await app.register(dbPlugin, { connectionString: process.env.DATABASE_URL! });
// app.db is now available everywhere (root and all children)
```

Plugin type	Use for	Wrap with fp?
Utility / connection	DB pools, config, auth decorators, shared hooks	Yes
Feature / routes	Route groups with their own hooks and prefix	No

8.4 Loading order and readiness

```
app.register(pluginA);
app.register(pluginB);            // starts after A has fully loaded
app.after(() => {
  // A and B are loaded here
});
await app.ready();                // everything loaded (listen() calls this for you)

// fp metadata can enforce dependencies and Fastify version
export default fp(plugin, {
  name: 'auth', fastify: '5.x', dependencies: ['db', '@fastify/jwt'],
});
```

9. Decorators

Decorators add properties or methods to the instance, request or reply. They are the Fastify way to share things (instead of attaching to `req` ad hoc), and they keep object shapes stable for V8 performance.

```
// Instance
app.decorate('config', { featureX: true });
app.decorate('sum', (a: number, b: number) => a + b);

// Request - initialize with null (or a getter), then set in a hook
app.decorateRequest('user', null);
app.addHook('preHandler', async (request) => {
  request.user = await loadUser(request.headers.authorization);
});

// Request with per-request object: use a getter (objects are not allowed directly)
app.decorateRequest('ctx', { getter() { return { startedAt: Date.now() }; } });

// Reply
app.decorateReply('ok', function (data: unknown) {
  return this.code(200).send({ success: true, data });
});

app.hasDecorator('config');       // true
```

TypeScript: declaration merging for decorators

```
declare module 'fastify' {
  interface FastifyInstance {
    config: { featureX: boolean };
    sum(a: number, b: number): number;
  }
  interface FastifyRequest { user: { id: string; role: string } | null }
  interface FastifyReply { ok(data: unknown): FastifyReply }
}
```

Watch out: Decorating the request/reply with a **shared object or array** means every request shares the same reference. Fastify v5 throws for this; initialize with `null` or use a getter.

9.1 Real-world example: a multi-tenant SaaS (plugins + decorators)

The situation. One deployment serves many companies: `acme.app.example.com`, `globex.app.example.com`. Every request must be tied to a tenant, each tenant sees only its data, and an admin area per tenant needs its own key.

Why it works well in Fastify:

- A shared `fp()` plugin resolves the tenant once and decorates the request.
- Encapsulation keeps the admin hook and decorator out of the public API.
- The structure is visible in the code: shared infrastructure, then feature plugins.

```
// src/multi-tenant/tenants.ts
import fp from 'fastify-plugin';

export interface Tenant {
  id: string;
  name: string;
  plan: 'free' | 'pro';
  adminKey: string;
}

export interface TenantStore {
  bySlug(slug: string): Tenant | undefined;
  projects(tenantId: string): string[];
  addProject(tenantId: string, name: string): void;
}
```

```
declare module 'fastify' {
  interface FastifyInstance {
    tenants: TenantStore;
  }
  interface FastifyRequest {  1
    tenant: Tenant | null;
  }
}

// fp(): shared infrastructure. Decorators and hooks added here are visible to the
// parent (the whole app), not encapsulated in this plugin
export default fp<{ store: TenantStore }>(  2
  async (app, opts) => {
    app.decorate('tenants', opts.store);
    app.decorateRequest('tenant', null); // null, never a shared object  3
```

1	Type the request decorator so every handler sees `request.tenant`.
2	`fp()` breaks encapsulation on purpose: the hook and decorators apply to the whole app.
3	Initialize with `null`: a shared object would leak between requests (Fastify 5 refuses it).

```
    // acme.app.example.com -> "acme"; X-Tenant header for API clients / tests
    app.addHook('onRequest', async (request, reply) => {
      const slug =
        (request.headers['x-tenant'] as string | undefined) ??
        request.hostname.split('.')[0];  4
      request.tenant = app.tenants.bySlug(slug) ?? null;
      if (!request.tenant) {
        return reply.code(404).send({ message: `Unknown workspace "${slug}"` });  5
      }
    });
  },
  { name: 'tenants' },
);
```

4	`acme.app.example.com` → `acme`; the header form is for API clients and tests.
5	Unknown tenants never reach a route.

```
// src/multi-tenant/app.ts
import Fastify, { FastifyPluginAsync } from 'fastify';
import { timingSafeEqual } from 'node:crypto';
import tenants, { TenantStore } from './tenants';

// Encapsulated: everything registered in here stays in here
const adminArea: FastifyPluginAsync = async (admin) => {  1
  const auditTrail: string[] = [];
  admin.decorate('audit', (entry: string) => {  2
    auditTrail.push(entry);
  });

  // Runs only for routes in this plugin (and its children)
  admin.addHook('onRequest', async (request, reply) => {  3
    const given = Buffer.from(String(request.headers['x-admin-key'] ?? ''));
    const expected = Buffer.from(request.tenant!.adminKey);
    if (given.length !== expected.length || !timingSafeEqual(given, expected)) {  4
      return reply.code(401).send({ message: 'Admin key required' });
    }
  });
```

1	Not wrapped in `fp()`: a separate context.
2	Exists only inside the admin context (and its children).
3	The admin-key check runs for admin routes only.
4	Constant-time comparison; each tenant has its own key.

```
  admin.post<{ Body: { name: string } }>('/projects', async (request, reply) => {
    admin.tenants.addProject(request.tenant!.id, request.body.name);
    admin.audit(`${request.tenant!.id}: created ${request.body.name}`);
    return reply.code(201).send({ ok: true });
  });
  admin.get('/audit', async () => auditTrail);
};

const publicApi: FastifyPluginAsync = async (api) => {
  api.get('/projects', async (request) => ({
    workspace: request.tenant!.name,
    projects: api.tenants.projects(request.tenant!.id), // only this tenant's data  5
  }));
  // Demonstrates encapsulation: the admin decorator does not exist here
  api.get('/debug/has-audit', async () => ({
    hasAudit: api.hasDecorator('audit'),  6
  }));
};
```

5 Data access is always scoped by the resolved tenant.

6 Proof of encapsulation: `false` here (see the test).

```
export async function buildTenantApp(store: TenantStore) {
  const app = Fastify();
  await app.register(tenants, { store }); // shared: every route has request.tenant  7
  await app.register(publicApi, { prefix: '/api' });
  await app.register(adminArea, { prefix: '/admin' });
  return app;
}

declare module 'fastify' {
  interface FastifyInstance {
    audit(entry: string): void;
  }
}
```

7 Registration order: shared infrastructure first, then feature plugins.

What the tests prove (`test/multi-tenant.test.ts`, all passing):

Test	Verified behaviour
the subdomain selects the tenant; each tenant sees only its data	Acme sees Website; Globex sees Rocket.
an unknown workspace is rejected by the shared hook	404.
the admin hook protects only the admin area	No key 401, other tenant's key 401, right key 201; audit entry recorded.
encapsulation: the admin decorator is invisible to public routes	`hasDecorator('audit')` is false in the public plugin; `tenants` is visible app-wide.

10. Hooks

10.1 Request/reply hooks

Hook	Signature (async)	Typical use
`onRequest`	`(req, reply)`	Authentication, request timing.
`preParsing`	`(req, reply, payload)`	Transform raw body stream.
`preValidation`	`(req, reply)`	Normalize input before schema check.
`preHandler`	`(req, reply)`	Authorization, load resources.
`preSerialization`	`(req, reply, payload)`	Wrap/transform the returned object.
`onSend`	`(req, reply, payload)`	Change serialized payload/headers.
`onResponse`	`(req, reply)`	Metrics, logging (response already sent).
`onError`	`(req, reply, error)`	Observe errors (cannot change response).
`onTimeout`	`(req, reply)`	Socket timed out.
`onRequestAbort`	`(req)`	Client closed the connection early.

```
// Global timing header
app.addHook('onRequest', async (request) => {
  request.log.debug({ url: request.url }, 'start');
});
app.addHook('onSend', async (request, reply, payload) => {
  reply.header('X-Response-Time', `${reply.elapsedTime.toFixed(1)}ms`);
  return payload;                          // must return (possibly modified) payload
});

// Envelope every JSON response
app.addHook('preSerialization', async (request, reply, payload) => {
  return { data: payload, requestId: request.id };
});

// Short-circuit: respond early from a hook
app.addHook('onRequest', async (request, reply) => {
  if (maintenanceMode) {
    return reply.code(503).send({ message: 'Maintenance' });   // stops the chain
  }
});
```

10.2 Route-level hooks

```
app.get('/admin', {
  onRequest: [app.authenticate],             // array or single function
  preHandler: async (request) => {
    if (request.user?.role !== 'admin') throw app.httpErrors.forbidden();
  },
}, async () => ({ secret: 42 }));
```

10.3 Application hooks

Hook	Runs when
`onRoute`	A route is registered (inspect/modify route options).
`onRegister`	A new encapsulated context is created.
`onReady`	All plugins loaded, before listening. Warm caches.
`onListen`	Server is listening.
`preClose` / `onClose`	During `app.close()` — close DB pools, flush queues.

```
app.addHook('onRoute', (route) => {
  app.log.info(`${route.method} ${route.url}`);   // print route table at boot
});
app.addHook('onReady', async () => { await warmCache(); });
app.addHook('onClose', async (instance) => { await instance.db.end(); });
```

Tip: Hooks follow encapsulation: a hook added inside a plugin runs only for routes in that plugin and its children. Use `app.printRoutes()` and `app.printPlugins()` to debug the tree.

10.4 Real-world example: production observability with hooks

The situation. Operations needs to trace a request across services, see server timing, audit every change, switch the API into maintenance mode, and be sure no password or token ever lands in the logs.

Why it works well in Fastify:

- Each concern is one hook at the right lifecycle stage.
- Pino redaction is declarative.
- `onResponse` work never delays the client.

```
// src/observability/app.ts
import Fastify from 'fastify';
import { randomUUID } from 'node:crypto';

export interface AuditEntry {
  requestId: string;
  method: string;
  url: string;
  status: number;
  ms: number;
}

export interface ObservabilityOptions {
  logSink: (line: string) => void; // stdout in production, an array in tests  1
  audit: (entry: AuditEntry) => void; // e.g. an audit table or Kafka topic
}
```

1 Logs go to any writable; tests capture them to assert on content.

```
export async function buildObservedApp({ logSink, audit }: ObservabilityOptions) {
  const app = Fastify({
    logger: {
      level: 'info',
      stream: { write: logSink },
      redact: {
        paths: ['headers.authorization', 'headers.cookie', 'body.password'],  2
        censor: '[REDACTED]',
      },
    },
    requestIdHeader: 'x-request-id', // reuse the ID from the gateway / caller...  3
    genReqId: () => randomUUID(), // ...or make a new one
    disableRequestLogging: true, // we log one structured line ourselves  4
  });
```

2	Redaction paths: secrets never reach log storage.
3	Correlate one request across gateway, API and downstream services.
4	Replace the two default lines (incoming/completed) with one richer line.

```
  let maintenance = false;
  app.decorate('setMaintenance', (on: boolean) => {
    maintenance = on;
  });

  // 1. Maintenance switch: answer before routing work starts (health stays up)
  app.addHook('onRequest', async (request, reply) => {  5
    if (maintenance && request.url !== '/health') {
      return reply
        .code(503)
        .header('retry-after', '120')
        .send({ message: 'Down for maintenance' });
    }
  });
```

5	Earliest hook: the maintenance switch answers before parsing or validation.

```
  // 2. Propagate the request ID and expose server timing to clients
  app.addHook('onSend', async (request, reply, payload) => {  6
    reply.header('x-request-id', request.id);
    reply.header('server-timing', `app;dur=${reply.elapsedTime.toFixed(1)}`);
    return payload;
  });

  // 3. After the response is sent: log + audit (never delays the client)
  app.addHook('onResponse', async (request, reply) => {  7
    request.log.info(
      {
        method: request.method,
        url: request.url,
        status: reply.statusCode,
        ms: Math.round(reply.elapsedTime),
      },
      'request completed',
    );
    if (request.method !== 'GET') {
      audit({
        requestId: request.id,
        method: request.method,
        url: request.url,
        status: reply.statusCode,
        ms: Math.round(reply.elapsedTime),
      });
    }
  });
```

6 onSend: add headers to every response; must return the payload.

7 onResponse runs after the client has the response: logging and auditing never slow it down.

```
  app.get('/health', async () => ({ status: 'ok' }));

  app.post<{ Body: { email: string; password: string } }>(
    '/login',
    async (request) => {
      // Logs show who tried, never the password or the Authorization header
      request.log.info(
        { headers: request.headers, body: request.body },  8
        'login attempt',
      );
      return { ok: true };
    },
  );

  return app;
}
```

8 Logged deliberately with secrets inside — the test proves they come out redacted.

```
declare module 'fastify' {
  interface FastifyInstance {
    setMaintenance(on: boolean): void;
  }
}
```

What the tests prove (`test/observability.test.ts`, all passing):

Test	Verified behaviour
the caller's request ID is propagated, server timing is exposed	Incoming `x-request-id` echoed; otherwise a UUID; `server-timing` present.
secrets are redacted; every log line carries the request ID	Authorization header and password show [REDACTED]; `hunter2` appears nowhere.
mutating requests are audited after the response	POST audited with status 200; GET not audited.
maintenance mode answers 503 early but keeps /health up	503 + Retry-After 120; health still 200; back to normal when switched off.

11. Error Handling

11.1 Throwing errors

```
app.get('/items/:id', async (request) => {
  const item = await repo.find(request.params.id);
  if (!item) {
    const err = new Error('Item not found') as Error & { statusCode: number };
    err.statusCode = 404;
    throw err;                              // -> 404 JSON response
  }
  return item;
});
```

11.2 @fastify/sensible - HTTP error helpers

```
npm install @fastify/sensible
```

```
import sensible from '@fastify/sensible';
await app.register(sensible);

app.get('/items/:id', async (request, reply) => {
  const item = await repo.find(request.params.id);
  if (!item) throw app.httpErrors.notFound('Item not found');   // or reply.notFound()
  return item;
});
// also: badRequest, unauthorized, forbidden, conflict, tooManyRequests, ...
// plus app.assert(cond, 400, 'msg'), request.forwarded(), reply.vary(), app.to(promise)
```

11.3 Custom error classes with @fastify/error

```
import createError from '@fastify/error';

export const InsufficientFunds = createError(
  'APP_INSUFFICIENT_FUNDS', 'Account %s has insufficient funds', 422);

throw new InsufficientFunds(accountId);
// 422 {"statusCode":422,"code":"APP_INSUFFICIENT_FUNDS","error":"Unprocessable Entity",
//      "message":"Account acc-1 has insufficient funds"}
```

11.4 Central error handler and 404 handler

```
import { STATUS_CODES } from 'node:http';

app.setErrorHandler((error, request, reply) => {
  if (error.validation) {                              // schema validation failure
    return reply.code(400).send({
      statusCode: 400, error: 'Bad Request', message: error.message,
      details: error.validation,
    });
  }
  const status = error.statusCode ?? 500;
  if (status >= 500) request.log.error({ err: error }, 'unhandled error');
  reply.code(status).send({
    statusCode: status,
    error: STATUS_CODES[status] ?? 'Error',        // 'Not Found', 'Unauthorized', ...
    message: status >= 500 ? 'Something went wrong' : error.message, // hide internals
  });
});

app.setNotFoundHandler((request, reply) => {
  reply.code(404).send({ statusCode: 404, error: 'Not Found',
                         message: `Route ${request.method} ${request.url} not found` });
});
```

Error and not-found handlers are also encapsulated: a plugin can set its own (e.g. an HTML error page for `/web` and JSON for `/api`).

11.5 Real-world example: bank transfers with RFC 9457 problem details

The situation. A banking API must return machine-readable errors that client apps can branch on (insufficient funds, unknown account), without leaking internals when something breaks.

Why it works well in Fastify:

- Domain errors are classes with codes; handlers just throw.
- One error handler and one not-found handler define the whole error contract.
- The same shape for validation errors, domain errors, 404s and 500s.

```
// src/problem-details/app.ts
import Fastify, { FastifyError } from 'fastify';
import createError from '@fastify/error';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';

// Domain errors: a stable code, a message template, an HTTP status
export const AccountNotFound = createError(  1
  'ACCOUNT_NOT_FOUND',
  'Account %s does not exist',
  404,
);
export const InsufficientFunds = createError(
  'INSUFFICIENT_FUNDS',
  'Account %s has %s available, transfer needs %s',
  422,
);
export const SameAccount = createError(
  'SAME_ACCOUNT',
  'Source and destination must differ',
  422,
);
```

1 `@fastify/error`: a stable machine code, a message template and the HTTP status in one class.

```
const DOCS = 'https://docs.bank.example/errors';

export async function buildBankApp(balances: Map<string, number>) {
  const app = Fastify({ logger: false }).withTypeProvider<TypeBoxTypeProvider>();

  // One place turns every error into RFC 9457 "problem details"
  app.setErrorHandler((error: FastifyError, request, reply) => {  2
    const status = error.statusCode ?? 500;
    const problem: Record<string, unknown> = {
      type:
        status >= 500
          ? 'about:blank'
          : `${DOCS}/${(error.code ?? 'error').toLowerCase()}`,
      title: status >= 500 ? 'Internal Server Error' : (error.code ?? 'Error'),  3
      status,
      detail:
        status >= 500 ? 'Something went wrong, it has been logged' : error.message,  4
      instance: request.url,
    };
    if (error.validation) {
      problem.type = `${DOCS}/validation`;
      problem.title = 'Invalid request';
      problem['invalid-params'] = error.validation.map((v) => ({  5
        name: v.instancePath.slice(1),
        reason: v.message,
      }));
    }
    if (status >= 500) request.log.error({ err: error }, 'unhandled error');
    reply.code(status).type('application/problem+json').send(problem);  6
  });
```

2	One error handler shapes every failure as RFC 9457 problem details.
3	Clients branch on the stable code, never on the human message.
4	5xx details (here a password in an error message!) stay in the logs.
5	Validation failures use the RFC's `invalid-params` extension.
6	The dedicated media type tells clients what they are parsing.

```
  app.setNotFoundHandler((request, reply) => {  7
    reply
      .code(404)
      .type('application/problem+json')
      .send({
        type: `${DOCS}/route-not-found`,
        title: 'Not Found',
        status: 404,
        detail: `${request.method} ${request.url} is not an API endpoint`,
        instance: request.url,
      });
  });

  app.post(
    '/transfers',
    {
      schema: {
        body: Type.Object({
          from: Type.String({ pattern: '^ACC-\\d{4}$' }),
          to: Type.String({ pattern: '^ACC-\\d{4}$' }),
          amountCents: Type.Integer({ minimum: 1 }),
        }),
      },
    },
    async (request, reply) => {
      const { from, to, amountCents } = request.body;
      if (from === to) throw new SameAccount();
      for (const acc of [from, to])
        if (!balances.has(acc)) throw new AccountNotFound(acc);
      const available = balances.get(from)!;
      if (available < amountCents)
        throw new InsufficientFunds(from, available, amountCents);  8
      balances.set(from, available - amountCents);
      balances.set(to, balances.get(to)! + amountCents);
      return reply.code(201).send({ from, to, amountCents });
    },
  );
```

7 Unknown routes return the same shape.

8 Handlers just throw domain errors; the mapping lives in one place.

```
  app.get('/boom', async () => {
    // simulates a bug
    throw new Error('Connection to ledger lost: password=hunter2');
  });

  return app;
}
```

What the tests prove (`test/problem-details.test.ts`, all passing):

Test	Verified behaviour
a valid transfer moves money	201; balances updated.
domain errors become RFC 9457 problem details	422 `application/problem+json` with type, title, status, detail, instance; unknown account 404.
validation errors list the invalid parameters	400 with `invalid-params`.
unexpected errors hide their details from clients	500 with a generic detail; the password in the error message is not sent.
unknown routes get a problem document too	404 route-not-found problem.

12. Content-Type Parsers

Fastify parses `application/json` and `text/plain` out of the box. Anything else returns **415 Unsupported Media Type** until you add a parser.

```
// URL-encoded forms
import formbody from '@fastify/formbody';
await app.register(formbody);

// Custom parser: YAML
import yaml from 'yaml';
app.addContentTypeParser('application/yaml', { parseAs: 'string' },
  (request, body, done) => {
    try { done(null, yaml.parse(body as string)); }
    catch (err) { done(err as Error, undefined); }
  });

// Raw buffer for webhook signature checks (e.g. Stripe)
app.register(async (hooks) => {
  hooks.addContentTypeParser('application/json', { parseAs: 'buffer' },
    (req, body, done) => done(null, body));
  hooks.post('/webhooks/stripe', async (req) => verifySignature(req.body as Buffer));
});
```

12.1 Real-world example: payment-provider webhooks

The situation. The payment provider POSTs events (payment succeeded, refund created) to the API. Requests must be authenticated by an HMAC signature over the raw body, replays must be refused, and retried deliveries processed only once.

Why it works well in Fastify:

- An encapsulated content-type parser keeps the raw body for this route only.
- Signature, freshness and idempotency are each a few lines.
- The rest of the app keeps normal JSON parsing.

```
// src/webhooks/app.ts
import Fastify, { FastifyPluginAsync } from 'fastify';
import { createHmac, timingSafeEqual } from 'node:crypto';

export interface PaymentEvent {
  id: string;
  type: 'payment.succeeded' | 'payment.failed' | 'refund.created';
  data: { paymentId: string; amountCents: number };
}

export interface WebhookOptions {
  secret: string; // shared with the payment provider
  toleranceSec?: number; // reject replays of old requests  1
  now?: () => number; // injectable clock (tests)
  onEvent: (event: PaymentEvent) => Promise<void>;
}
```

1 A captured request replayed later is rejected by its timestamp.

```
// Header format used by Stripe-style providers: "t=<unix>,v1=<hex hmac>"
export function sign(secret: string, timestamp: number, rawBody: string) {  2
  const mac = createHmac('sha256', secret)
    .update(`${timestamp}.${rawBody}`)
    .digest('hex');
  return `t=${timestamp},v1=${mac}`;
}

const webhooks =
  (opts: WebhookOptions): FastifyPluginAsync =>
  async (hooks) => {
    const seen = new Set<string>(); // a DB unique index in production  3
    const now = opts.now ?? (() => Date.now() / 1000);

    // The signature covers the exact bytes sent, so keep the body as a Buffer here.
    // Encapsulated: the rest of the app keeps the normal JSON parser.
    hooks.addContentTypeParser(  4
      'application/json',
      { parseAs: 'buffer' },
      (request, body, done) => done(null, body),  5
    );
```

2	The provider's algorithm: HMAC-SHA256 over `timestamp.rawBody`.
3	Providers deliver at least once: remember processed event IDs.
4	Inside this plugin only, JSON bodies stay raw `Buffers`...
5	...because re-serializing parsed JSON changes the bytes and breaks the signature (the test proves it).

```
    hooks.post('/payments', async (request, reply) => {
      const raw = request.body as Buffer;
      const header = String(request.headers['x-signature'] ?? '');
      const parts = Object.fromEntries(
        header.split(',').map((kv) => kv.split('=')),
      );
      const timestamp = Number(parts.t);

      if (!timestamp || Math.abs(now() - timestamp) > (opts.toleranceSec ?? 300)) {  6
        return reply.code(401).send({ message: 'Stale or missing timestamp' });
      }
      const expected = Buffer.from(
        sign(opts.secret, timestamp, raw.toString()).split('v1=')[1],
      );
      const given = Buffer.from(String(parts.v1 ?? ''));
      if (given.length !== expected.length || !timingSafeEqual(given, expected)) {  7
        return reply.code(401).send({ message: 'Invalid signature' });
      }
```

6	Check freshness first: cheap, and blocks replays.
7	Constant-time comparison prevents timing attacks on the signature.

```
      const event = JSON.parse(raw.toString()) as PaymentEvent;
      // The provider retries deliveries: acknowledge duplicates, don't re-process
      if (seen.has(event.id)) return { received: true, duplicate: true };  8
      await opts.onEvent(event);
      seen.add(event.id);
      return { received: true, duplicate: false };
    });
  };

export async function buildWebhookApp(opts: WebhookOptions) {
  const app = Fastify();
  await app.register(webhooks(opts), { prefix: '/webhooks' });
  // A normal JSON route elsewhere in the app: still gets parsed objects
  app.post('/echo', async (request) => ({
    type: typeof request.body,
    body: request.body,
  }));
  return app;
}
```

8 Duplicates get 200 so the provider stops retrying.

```
// test/webhooks.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { buildWebhookApp, sign, PaymentEvent } from '../src/webhooks/app';

const SECRET = 'whsec_test_5f2c1a';
const NOW = 1_790_000_000;
const handled: string[] = [];

let app: Awaited<ReturnType<typeof buildWebhookApp>>;
before(async () => {
  app = await buildWebhookApp({
    secret: SECRET,
    now: () => NOW,
    onEvent: async (e) => {
      handled.push(`${e.type}:${e.data.paymentId}`);
    },
  });
});
after(() => app.close());
```

```
const event: PaymentEvent = {
  id: 'evt_1',
  type: 'payment.succeeded',
  data: { paymentId: 'pay_42', amountCents: 4999 },
};

const deliver = (body: string, signature: string) =>
  app.inject({
    method: 'POST',
    url: '/webhooks/payments',
    payload: body,
    headers: { 'content-type': 'application/json', 'x-signature': signature },
  });

test('a correctly signed event is accepted and handled once', async () => {
  const body = JSON.stringify(event);
  const res = await deliver(body, sign(SECRET, NOW, body));  1
  assert.equal(res.statusCode, 200);
  assert.deepEqual(res.json(), { received: true, duplicate: false });
  assert.deepEqual(handled, ['payment.succeeded:pay_42']);
});
```

1 Sign exactly like the provider; a fixed clock makes the test deterministic.

```
test('a retried delivery is acknowledged, not re-processed', async () => {
  const body = JSON.stringify(event);
  const res = await deliver(body, sign(SECRET, NOW, body));
  assert.deepEqual(res.json(), { received: true, duplicate: true });
  assert.equal(handled.length, 1);
});

test('a tampered body is rejected (the signature covers the raw bytes)', async () => {
  const body = JSON.stringify(event);
  const tampered = body.replace('4999', '1');
  assert.equal((await deliver(tampered, sign(SECRET, NOW, body))).statusCode, 401);
  // Same JSON, different bytes (whitespace) also fails: never re-serialize
  const reformatted = JSON.stringify(event, null, 2);  2
  assert.equal(
    (await deliver(reformatted, sign(SECRET, NOW, body))).statusCode,
    401,
  );
});
```

2 Same data, different bytes: rejected.

```
test('a replayed request older than 5 minutes is rejected', async () => {
  const body = JSON.stringify({ ...event, id: 'evt_2' });
  const res = await deliver(body, sign(SECRET, NOW - 301, body));
  assert.equal(res.statusCode, 401);
  assert.equal(res.json().message, 'Stale or missing timestamp');
});

test('the raw-body parser is encapsulated: other routes parse JSON', async () => {
  const res = await app.inject({ method: 'POST', url: '/echo', payload: { a: 1 } });
  assert.deepEqual(res.json(), { type: 'object', body: { a: 1 } });
});
```

What the tests prove (`test/webhooks.test.ts`, all passing):

Test	Verified behaviour
a correctly signed event is accepted and handled once	200, handled.
a retried delivery is acknowledged, not re-processed	200 duplicate: true; handler not called again.
a tampered body is rejected (the signature covers the raw bytes)	Changed amount → 401; re-formatted identical JSON → 401.
a replayed request older than 5 minutes is rejected	401 stale timestamp.
the raw-body parser is encapsulated: other routes parse JSON	`/echo` receives an object.

13. Logging with Pino

```
const app = Fastify({
  logger: {
    level: process.env.LOG_LEVEL ?? 'info',
    redact: ['req.headers.authorization', 'req.headers.cookie'],
    transport: process.env.NODE_ENV === 'production'
      ? undefined                                    // raw JSON in prod
      : { target: 'pino-pretty', options: { translateTime: 'HH:MM:ss' } },
  },
  genReqId: () => crypto.randomUUID(),
  requestIdHeader: 'x-request-id',                   // reuse incoming ID if present
  disableRequestLogging: false,
});

app.get('/orders/:id', async (request) => {
  request.log.info({ orderId: request.params }, 'fetching order');  // includes reqId
  return {};
});
```

- Levels: `fatal, error, warn, info, debug, trace`.
- Use `request.log` inside handlers so every line carries the request ID.
- To pass your own pino instance use `loggerInstance: pino(...)` (v5 change).
- Pino logs are JSON — ship them to ELK, Loki, Datadog, CloudWatch as-is.

Tip: See the observability example in 10.4 for redaction, request-ID propagation and capturing logs in tests.

Part III — The Plugin Ecosystem

14. Configuration with @fastify/env

```
npm install @fastify/env
```

```
import fp from 'fastify-plugin';
import fastifyEnv from '@fastify/env';

const schema = {
  type: 'object',
  required: ['DATABASE_URL', 'JWT_SECRET'],
  properties: {
    PORT: { type: 'number', default: 3000 },
    HOST: { type: 'string', default: '0.0.0.0' },
    DATABASE_URL: { type: 'string' },
    JWT_SECRET: { type: 'string', minLength: 32 },
  },
};

declare module 'fastify' {
  interface FastifyInstance {
    config: { PORT: number; HOST: string; DATABASE_URL: string; JWT_SECRET: string };
  }
}

export default fp(async (app) => {
  await app.register(fastifyEnv, { schema, dotenv: true });   // reads .env too
}, { name: 'env' });
// Boot fails with a clear error if a required variable is missing or invalid.
```

15. Databases

15.1 PostgreSQL with @fastify/postgres

```
import postgres from '@fastify/postgres';
await app.register(postgres, { connectionString: app.config.DATABASE_URL });

app.get('/users/:id', async (request) => {
  const { rows } = await app.pg.query(
    'SELECT id, email, name FROM users WHERE id = $1', [request.params.id]);
  if (!rows[0]) throw app.httpErrors.notFound();
  return rows[0];
});

// Transactions
app.post('/transfer', async (request) => {
  return app.pg.transact(async (client) => {
    await client.query('UPDATE accounts SET bal = bal - $1 WHERE id = $2', [amt, from]);
    await client.query('UPDATE accounts SET bal = bal + $1 WHERE id = $2', [amt, to]);
    return { ok: true };
  });
});
```

15.2 Any ORM (Prisma, Drizzle, Knex, Mongoose) via a plugin

```
import fp from 'fastify-plugin';
import { PrismaClient } from '@prisma/client';

declare module 'fastify' { interface FastifyInstance { prisma: PrismaClient } }

export default fp(async (app) => {
  const prisma = new PrismaClient();
  await prisma.$connect();
  app.decorate('prisma', prisma);
  app.addHook('onClose', async () => { await prisma.$disconnect(); });
}, { name: 'prisma' });

// usage: const user = await app.prisma.user.findUnique({ where: { id } });
```

Other official plugins: `@fastify/mongodb`, `@fastify/mysql`, `@fastify/redis`, `@fastify/elasticsearch`.

15.3 Real-world example: inventory reservations in SQL

The situation. Checkout reserves stock for every line of an order. If any line is short, nothing may be reserved; two simultaneous checkouts must never oversell.

Why it works well in Fastify:

- A database plugin owns the connection lifecycle (open at boot, close on shutdown).
- A transaction plus conditional UPDATEs make the reservation atomic.
- Domain errors map to precise HTTP codes (409, 404).

```
// src/inventory/db.ts
import fp from 'fastify-plugin';
import { DatabaseSync } from 'node:sqlite';  1

declare module 'fastify' {
  interface FastifyInstance {
    db: DatabaseSync;
  }
}

const MIGRATIONS = [
  `CREATE TABLE products (
     sku TEXT PRIMARY KEY,
     name TEXT NOT NULL,
     stock INTEGER NOT NULL CHECK (stock >= 0)  2
   )`,
  `CREATE TABLE orders (
     id INTEGER PRIMARY KEY AUTOINCREMENT,
     customer TEXT NOT NULL
   )`,
  `CREATE TABLE order_lines (
     order_id INTEGER NOT NULL REFERENCES orders(id),
     sku TEXT NOT NULL REFERENCES products(sku),
     qty INTEGER NOT NULL CHECK (qty > 0)
   )`,
];
```

1. Node's built-in SQLite (no install). The plugin pattern is identical for Postgres, MySQL or an ORM.
2. The database enforces the invariant too.

```
// Database connection as a shared (fp) plugin: opened at boot, closed on shutdown.
// Same shape as @fastify/postgres or a Prisma plugin.
export default fp<{ filename: string }>(
  async (app, opts) => {
    const db = new DatabaseSync(opts.filename);
    db.exec('PRAGMA foreign_keys = ON');
    for (const sql of MIGRATIONS) db.exec(sql);  3
    app.decorate('db', db);
    app.addHook('onClose', async () => db.close());  4
  },
  { name: 'db' },
);
```

3 Migrations at boot (real projects use a migration tool).

4 Closed on `app.close()` — verified by the last test.

```
// src/inventory/app.ts
import Fastify from 'fastify';
import createError from '@fastify/error';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import db from './db';

export const OutOfStock = createError(
  'OUT_OF_STOCK',
  'Only %s of %s left, %s requested',
  409,
);
export const UnknownProduct = createError('UNKNOWN_PRODUCT', 'No product %s', 404);

const Line = Type.Object({ sku: Type.String(), qty: Type.Integer({ minimum: 1 }) });
```

```
export async function buildInventoryApp(filename = ':memory:') {
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();
  await app.register(db, { filename });

  app.post(
    '/products',
    {
      schema: {
        body: Type.Object({
          sku: Type.String(),
          name: Type.String(),
          stock: Type.Integer({ minimum: 0 }),
        }),
      },
    },
    async (request, reply) => {
      const { sku, name, stock } = request.body;
      app.db
        .prepare('INSERT INTO products (sku, name, stock) VALUES (?, ?, ?)')
        .run(sku, name, stock); // parameters, never string concatenation  1
      return reply.code(201).send({ sku, name, stock });
    },
  );
```

1 Bound parameters: no SQL injection.

```
  app.get(
    '/products/:sku',
    {
      schema: { params: Type.Object({ sku: Type.String() }) },
    },
    async (request) => {
      const row = app.db
        .prepare('SELECT sku, name, stock FROM products WHERE sku = ?')
        .get(request.params.sku);
      if (!row) throw new UnknownProduct(request.params.sku);
      return row;
    },
  );

  // Checkout: reserve stock for every line, or for none of them
  app.post(
    '/orders',
    {
      schema: {
        body: Type.Object({
          customer: Type.String(),
          lines: Type.Array(Line, { minItems: 1 }),
        }),
      },
    },
    async (request, reply) => {
      const { customer, lines } = request.body;
      const reserve = app.db.prepare(
        'UPDATE products SET stock = stock - ? WHERE sku = ? AND stock >= ?',
      );
      const find = app.db.prepare('SELECT stock FROM products WHERE sku = ?');

      app.db.exec('BEGIN IMMEDIATE'); // take the write lock up front  2
      try {
        for (const { sku, qty } of lines) {
          // Conditional UPDATE: check + decrement in one statement (no race)
          if (reserve.run(qty, sku, qty).changes === 0) {  3
            const row = find.get(sku) as { stock: number } | undefined;
            if (!row) throw new UnknownProduct(sku);
            throw new OutOfStock(row.stock, sku, qty);  4
          }
        }
        const { lastInsertRowid } = app.db
          .prepare('INSERT INTO orders (customer) VALUES (?)')
          .run(customer);
        const addLine = app.db.prepare(
          'INSERT INTO order_lines (order_id, sku, qty) VALUES (?, ?, ?)',
        );
        for (const { sku, qty } of lines) addLine.run(lastInsertRowid, sku, qty);
        app.db.exec('COMMIT');
        return reply.code(201).send({ orderId: Number(lastInsertRowid) });
      } catch (err) {
        app.db.exec('ROLLBACK'); // earlier lines are released again  5
        throw err;
      }
    },
  );

  return app;
}
```

2	One transaction for the whole order.
3	`UPDATE ... WHERE stock >= ?`: the check and the decrement happen in one statement, so two checkouts can't oversell.
4	A 409 with a precise message: `Only 1 of TEE left, 5 requested`.
5	All or nothing: the MUG reserved by the first line is given back.

What the tests prove (`test/inventory.test.ts`, all passing):

Test	Verified behaviour
an order reserves stock for every line	MUG 10 → 7, TEE 2 → 1.
one short line rolls back the whole order	409 "Only 1 of TEE left, 5 requested"; MUG unchanged at 7.
unknown products are a 404, and nothing is reserved	MUG unchanged.
the database is closed when the app closes	Using `app.db` after `app.close()` throws.

16. Authentication

16.1 JWT with @fastify/jwt

```
npm install @fastify/jwt
```

```
import jwt from '@fastify/jwt';
import type { FastifyRequest } from 'fastify';

declare module '@fastify/jwt' {
  interface FastifyJWT {
    payload: { sub: string; role: 'user' | 'admin' };   // what you sign
    user: { sub: string; role: 'user' | 'admin' };      // request.user after verify
  }
}
declare module 'fastify' {
  interface FastifyInstance { authenticate(req: FastifyRequest): Promise<void> }
}

await app.register(jwt, { secret: app.config.JWT_SECRET, sign: { expiresIn: '15m' } });

app.decorate('authenticate', async (request: FastifyRequest) => {
  try { await request.jwtVerify(); }
  catch { throw app.httpErrors.unauthorized('Invalid or expired token'); }
});

app.post('/login', async (request) => {
  const user = await checkCredentials(request.body);
  return { token: app.jwt.sign({ sub: user.id, role: user.role }) };
});

app.get('/me', { onRequest: [app.authenticate] }, async (request) => request.user);
```

16.2 Combining strategies with @fastify/auth

```
import fastifyAuth from '@fastify/auth';
await app.register(fastifyAuth);

app.get('/reports', {
  preHandler: app.auth([app.verifyJwt, app.verifyApiKey]),   // either passes
  // app.auth([a, b], { relation: 'and' })                   // both must pass
}, getReports);
```

16.3 Cookies and sessions

```
import cookie from '@fastify/cookie';
import session from '@fastify/session';

await app.register(cookie);
await app.register(session, {
  secret: app.config.SESSION_SECRET,         // >= 32 chars
  cookie: { secure: true, httpOnly: true, sameSite: 'lax', maxAge: 86_400_000 },
});

app.post('/login', async (request) => {
  request.session.userId = user.id;          // stored server-side
  return { ok: true };
});
// Default store is in-memory: use a Redis/DB store in production.
```

16.4 Real-world example: login, roles and refresh-token rotation

The situation. A B2B dashboard: people log in (short-lived access tokens + refresh tokens), admins can delete users, analysts see revenue, and a nightly ETL job reads reports with an API key.

Why it works well in Fastify:

- Authentication in `onRequest`, authorization in `preHandler`: 401 vs 403 fall out naturally.
- `@fastify/auth` combines JWT and API keys on one route.
- Refresh rotation with reuse detection stops stolen refresh tokens.

```
// src/auth-rbac/app.ts
import Fastify, { FastifyRequest } from 'fastify';
import jwt from '@fastify/jwt';
import fastifyAuth from '@fastify/auth';
import sensible from '@fastify/sensible';
import { Type, TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import { randomUUID, timingSafeEqual } from 'node:crypto';

export type Role = 'admin' | 'analyst' | 'user';
interface Account {
  id: string;
  email: string;
  password: string;
  roles: Role[];
}
```

```
declare module '@fastify/jwt' {
  interface FastifyJWT {
    payload: { sub: string; roles: Role[] };
    user:  1
      { sub: string; roles: Role[] } | { sub: string; roles: Role[]; apiKey: true };
  }
}

export interface AuthOptions {
  secret: string;
  accounts: Account[];
  apiKeys: string[];
}

export async function buildAuthApp({ secret, accounts, apiKeys }: AuthOptions) {
  const app = Fastify().withTypeProvider<TypeBoxTypeProvider>();
  await app.register(sensible);
  await app.register(jwt, { secret, sign: { expiresIn: '15m' } }); // short-lived  2
  await app.register(fastifyAuth);
```

1 `request.user` is either a person (JWT) or a service (API key).

2 Short-lived access tokens limit the damage of a leaked token.

```
  // Refresh tokens: opaque, stored server-side, rotated on every use.
  // family = one login session; reusing a rotated token revokes the whole family.  3
  const refreshTokens = new Map<
    string,
    { sub: string; family: string; used: boolean }
  >();
  const revokedFamilies = new Set<string>();

  const issue = (account: Account, family: string = randomUUID()) => {
    const refreshToken = randomUUID();
    refreshTokens.set(refreshToken, { sub: account.id, family, used: false });
    return {
      accessToken: app.jwt.sign({ sub: account.id, roles: account.roles }),
      refreshToken,
    };
  };
```

3 Refresh-token rotation with reuse detection (OAuth 2.0 security BCP).

```
  const verifyJwt = async (request: FastifyRequest) => {
    await request.jwtVerify();
  };
  const verifyApiKey = async (request: FastifyRequest) => {  4
    const key = Buffer.from(String(request.headers['x-api-key'] ?? ''));
    const ok = apiKeys.some(
      (k) => k.length === key.length && timingSafeEqual(Buffer.from(k), key),
    );
    if (!ok) throw app.httpErrors.unauthorized('Invalid API key');
    request.user = { sub: 'service', roles: ['analyst'], apiKey: true };
  };
  const requireRole = (role: Role) => async (request: FastifyRequest) => {  5
    if (!request.user.roles.includes(role)) {
      throw app.httpErrors.forbidden(`Requires role ${role}`);
    }
  };
```

4 Machine clients use API keys, compared in constant time.

5 Authorization is a separate, reusable step after authentication.

```
app.post(
  '/auth/login',
  {
    schema: {
      body: Type.Object({ email: Type.String(), password: Type.String() }),
    },
  },
  async (request) => {
    const account = accounts.find((a) => a.email === request.body.email);
    // Real code: compare a password hash (argon2/scrypt), not the plain text
    if (!account || account.password !== request.body.password) {
      throw app.httpErrors.unauthorized('Invalid credentials');
    }
    return issue(account);
  },
);
```

```
app.post(
  '/auth/refresh',
  {
    schema: { body: Type.Object({ refreshToken: Type.String() }) },
  },
  async (request) => {
    const stored = refreshTokens.get(request.body.refreshToken);
    if (!stored || revokedFamilies.has(stored.family)) {
      throw app.httpErrors.unauthorized('Invalid refresh token');
    }
    if (stored.used) {  6
      // stolen token being replayed
      revokedFamilies.add(stored.family);
      throw app.httpErrors.unauthorized(
        'Refresh token reuse detected, session revoked',
      );
    }
    stored.used = true;
    return issue(
      accounts.find((a) => a.id === stored.sub)!,
      stored.family,
    );
  },
);
```

6 A used refresh token presented again means it was copied: revoke the whole session.

```
  app.get('/me', { onRequest: [verifyJwt] }, async (request) => request.user);

  app.delete(
    '/users/:id',
    {
      onRequest: [verifyJwt],
      preHandler: [requireRole('admin')], // authorization after authN  7
    },
    async (_request, reply) => reply.code(204).send(),
  );

  // Dashboards (JWT) and the nightly ETL job (API key) share one endpoint
  app.get(
    '/reports/revenue',
    {
      preHandler: app.auth([verifyJwt, verifyApiKey]), // either is enough  8
    },
    async (request) => {
      await requireRole('analyst')(request);
      return { revenueCents: 12_345_600, viewer: request.user.sub };
    },
  );

  return app;
}
```

7 `onRequest` authenticates (401), `preHandler` authorizes (403).

8 `@fastify/auth`: the first strategy that succeeds wins.

```
// test/auth-rbac.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { buildAuthApp } from '../src/auth-rbac/app';

let app: Awaited<ReturnType<typeof buildAuthApp>>;
before(async () => {
  app = await buildAuthApp({
    secret: 'test-secret-that-is-long-enough-1234567',
    accounts: [
      { id: 'u1', email: 'ann@corp.example', password: 'pw-ann', roles: ['user'] },
      {
        id: 'u2',
        email: 'root@corp.example',
        password: 'pw-root',
        roles: ['admin', 'analyst'],
      },
    ],
    apiKeys: ['etl-job-key-0001'],
  });
});
after(() => app.close());
```

```
const login = async (email: string, password: string) =>
  (
    await app.inject({
      method: 'POST',
      url: '/auth/login',
      payload: { email, password },
    })
  ).json();
const bearer = (token: string) => ({ authorization: `Bearer ${token}` });

test('authentication: no token 401, valid token identifies the user', async () => {
  assert.equal((await app.inject('/me')).statusCode, 401);
  const { accessToken } = await login('ann@corp.example', 'pw-ann');
  const me = await app.inject({ url: '/me', headers: bearer(accessToken) });
  assert.equal(me.json().sub, 'u1');
});
```

```
test('authorization: users get 403 on admin routes, admins get through', async () => {
  const ann = await login('ann@corp.example', 'pw-ann');
  const root = await login('root@corp.example', 'pw-root');
  const del = (token: string) =>
    app.inject({ method: 'DELETE', url: '/users/u9', headers: bearer(token) });
  assert.equal((await del(ann.accessToken)).statusCode, 403);
  assert.equal((await del(root.accessToken)).statusCode, 204);
});

test('refresh tokens rotate; reusing an old one revokes the session', async () => {
  const first = await login('ann@corp.example', 'pw-ann');
  const refresh = (refreshToken: string) =>
    app.inject({ method: 'POST', url: '/auth/refresh', payload: { refreshToken } });

  const second = (await refresh(first.refreshToken)).json();
  assert.ok(second.accessToken);
  assert.notEqual(second.refreshToken, first.refreshToken);

  const replay = await refresh(first.refreshToken); // attacker's copy  1
  assert.equal(replay.statusCode, 401);
  assert.match(replay.json().message, /reuse detected/);
  assert.equal(
    (await refresh(second.refreshToken)).statusCode,
    401,
    'the legitimate session is revoked too',  2
  );
});

test('reports accept a JWT with the analyst role OR a service API key', async () => {
  const root = await login('root@corp.example', 'pw-root');
  const ann = await login('ann@corp.example', 'pw-ann');
  const get = (headers: Record<string, string>) =>
    app.inject({ url: '/reports/revenue', headers });
  assert.equal((await get(bearer(root.accessToken))).statusCode, 200);
  assert.equal((await get(bearer(ann.accessToken))).statusCode, 403);
  const etl = await get({ 'x-api-key': 'etl-job-key-0001' });
  assert.equal(etl.json().viewer, 'service');
  assert.equal((await get({ 'x-api-key': 'guess' })).statusCode, 401);
});
```

1	An attacker replays the old refresh token...
2	...and the session is dead for both: the user logs in again, the attacker is out.

What the tests prove (`test/auth-rbac.test.ts`, all passing):

Test	Verified behaviour
authentication: no token 401, valid token identifies the user	/me → u1.
authorization: users get 403 on admin routes, admins get through	403 vs 204.
refresh tokens rotate; reusing an old one revokes the session	New pair on refresh; replaying the old token → 401 and the newer token is revoked too.
reports accept a JWT with the analyst role OR a service API key	Admin+analyst 200, plain user 403, API key 200, wrong key 401.

17. Security Plugins

```
npm install @fastify/helmet @fastify/cors @fastify/rate-limit
```

```
import helmet from '@fastify/helmet';
import cors from '@fastify/cors';
import rateLimit from '@fastify/rate-limit';

await app.register(helmet);                                  // secure HTTP headers
await app.register(cors, {
  origin: ['https://app.example.com'],                       // never '*' with credentials
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE'],
});
await app.register(rateLimit, {
  max: 100,                                                  // per IP per window
  timeWindow: '1 minute',
  // redis: new Redis(),                                     // share limits across instances
});

// Stricter limit for one route
app.post('/login', { config: { rateLimit: { max: 5, timeWindow: '1 minute' } } }, login);
```

Security checklist

- Validate every input with schemas; use `additionalProperties: false`.
- Always define response schemas to avoid leaking fields.
- Keep `bodyLimit` small; raise it only on upload routes.
- Set `trustProxy` correctly so `request.ip` and rate limits use the real client IP.
- Hide 5xx error details from clients (Chapter 11.4); redact secrets in logs.
- Keep secrets in environment variables, validated by `@fastify/env`.

17.1 Real-world example: hardening a public API

The situation. A storefront's API runs behind a load balancer and is called from the shop's own web app. Bots try password lists against the login.

Why it works well in Fastify:

- Helmet, CORS and rate limiting are one `register()` each.
- Per-route options tighten limits exactly where needed.
- The tests caught a limiter that would have locked out whole offices.

```
// src/security/app.ts
import Fastify from 'fastify';
import helmet from '@fastify/helmet';
import cors from '@fastify/cors';
import rateLimit from '@fastify/rate-limit';

export async function buildSecureApp({
  allowedOrigins,
}: {
  allowedOrigins: string[];
}) {
  const app = Fastify({
    trustProxy: true, // behind a load balancer: request.ip from X-Forwarded-For  1
    bodyLimit: 64 * 1024, // 64 KB is plenty for JSON APIs
  });
```

1 Without it every client looks like the load balancer, and one IP-based limit blocks everybody.

```
  await app.register(helmet); // HSTS, nosniff, frame-options, CSP...  2

  await app.register(cors, {
    origin: allowedOrigins, // exact allowlist, never "*" with credentials  3
    credentials: true,
    methods: ['GET', 'POST', 'PUT', 'DELETE'],
    maxAge: 600, // browsers may cache the preflight for 10 minutes
  });

  // Global default: generous limit per client IP
  await app.register(rateLimit, { max: 300, timeWindow: '1 minute' });  4

  // Brute-force protection: 5 attempts per IP + username per 15 minutes
  app.post<{ Body: { username: string; password: string } }>(
    '/login',
    {
      config: {
        rateLimit: {
          max: 5,
          timeWindow: '15 minutes',
          // Default hook is onRequest, where the body is not parsed yet:
          // count in preHandler so the key can include the username
          hook: 'preHandler',  5
          keyGenerator: (request) =>
            `${request.ip}:${(request.body as { username?: string }).username}`,  6
        },
      },
    },
    async () => ({ ok: false, message: 'Invalid credentials' }),
  );
```

2	Secure headers on every response, `x-powered-by` removed.
3	Only the storefront's origin gets CORS headers; other sites get none.
4	A global safety net per client IP.
5	Found by the test: in the default `onRequest` hook the body isn't parsed, so the key had no username and blocked **every** user on that IP.
6	Key = IP + username: an attacker can't lock out other users.

```
  // Small endpoint, tiny body limit: rejected before the handler runs
  app.post('/feedback', { bodyLimit: 1024 }, async () => ({ thanks: true }));  7

  app.get('/whoami', async (request) => ({ ip: request.ip }));
  return app;
}
```

7	Per-route body limit: 413 before any parsing work.

What the tests prove (`test/security.test.ts`, all passing):

Test	Verified behaviour
security headers are set on every response	HSTS, nosniff, CSP present; no `x-powered-by`.
CORS: the storefront may call the API, other sites may not	Allowed origin echoed with credentials; other origin gets no CORS headers.
behind a proxy, request.ip is the real client	`203.0.113.9`.
brute force: the 6th login for one user from one IP gets 429	5 × 200 then 429 + Retry-After; another user and another IP still 200.
oversized bodies get 413 before reaching the handler	2 KB to a 1 KB route.

18. OpenAPI / Swagger Docs

```
npm install @fastify/swagger @fastify/swagger-ui
```

```
import swagger from '@fastify/swagger';
import swaggerUi from '@fastify/swagger-ui';

// Register BEFORE your routes so they are picked up
await app.register(swagger, {
  openapi: {
    info: { title: 'Tasks API', version: '1.0.0' },
    components: {
      securitySchemes: { bearerAuth: { type: 'http', scheme: 'bearer' } },
    },
  },
});
await app.register(swaggerUi, { routePrefix: '/docs' });

app.get('/tasks', {
  schema: {
    tags: ['tasks'],
    summary: 'List tasks',
    security: [{ bearerAuth: [] }],
    response: { 200: { type: 'array', items: { $ref: 'task#' } } },
  },
}, listTasks);
// Browse http://localhost:3000/docs  - JSON at /docs/json
```

Your validation schemas *are* your API documentation — they can never drift apart.

19. Static Files, Uploads & WebSockets

19.1 Static files

```
import fastifyStatic from '@fastify/static';
import path from 'node:path';

await app.register(fastifyStatic, {
  root: path.join(__dirname, '..', 'public'),
  prefix: '/public/',                          // /public/logo.png
});
app.get('/', async (req, reply) => reply.sendFile('index.html'));
```

19.2 File uploads with @fastify/multipart

```
import multipart from '@fastify/multipart';
import { createWriteStream } from 'node:fs';
import { pipeline } from 'node:stream/promises';

await app.register(multipart, { limits: { fileSize: 10 * 1024 * 1024, files: 1 } });

app.post('/avatar', async (request, reply) => {
  const data = await request.file();          // streaming - no full buffering
  if (!data) throw app.httpErrors.badRequest('file required');
  if (!['image/png', 'image/jpeg'].includes(data.mimetype)) {
    throw app.httpErrors.unsupportedMediaType();
  }
  await pipeline(data.file, createWriteStream(`./uploads/${crypto.randomUUID()}`));
  if (data.file.truncated) throw app.httpErrors.payloadTooLarge();
  reply.code(201);
  return { filename: data.filename };
});
// curl -F "file=@me.png" localhost:3000/avatar
```

19.3 WebSockets

```
import websocket from '@fastify/websocket';
await app.register(websocket);

const clients = new Set<import('ws').WebSocket>();

app.get('/chat', { websocket: true }, (socket, request) => {
  clients.add(socket);
  socket.on('message', (msg) => {
    for (const c of clients) c.send(`${request.id}: ${msg}`);   // broadcast
  });
  socket.on('close', () => clients.delete(socket));
});
// Normal hooks (onRequest auth, etc.) run on the upgrade request.
```

19.4 Real-world example: profile picture uploads

The situation. Users upload avatars. Attackers upload scripts named `cat.png`, huge files, and names like `../../etc/passwd.png`.

Why it works well in Fastify:

- Streaming multipart: files never sit fully in memory.
- Content sniffing (magic bytes) instead of trusting headers.
- Random server-side file names.

```
// src/uploads/app.ts
import Fastify from 'fastify';
import multipart from '@fastify/multipart';
import sensible from '@fastify/sensible';
import { createWriteStream } from 'node:fs';
import { mkdir, unlink } from 'node:fs/promises';
import { pipeline } from 'node:stream/promises';
import { randomUUID } from 'node:crypto';
import path from 'node:path';

// File signatures ("magic bytes"): trust the content, not the client's Content-Type
const SIGNATURES: Record<string, Buffer> = {  1
  png: Buffer.from([0x89, 0x50, 0x4e, 0x47]),
  jpg: Buffer.from([0xff, 0xd8, 0xff]),
};
const detect = (head: Buffer) =>
  Object.keys(SIGNATURES).find((ext) =>
    head.subarray(0, SIGNATURES[ext].length).equals(SIGNATURES[ext]),
  );
```

1 Magic bytes identify the real file type; the client's Content-Type and extension are just claims.

```
export async function buildUploadApp({
  dir,
  maxBytes = 2 * 1024 * 1024,
}: {
  dir: string;
  maxBytes?: number;
}) {
  const app = Fastify();
  await app.register(sensible);
  await app.register(multipart, {
    limits: { fileSize: maxBytes, files: 1, fields: 5 },  2
  });
  await mkdir(dir, { recursive: true });

  app.post('/me/avatar', async (request, reply) => {
    const file = await request.file();
    if (!file)
      throw app.httpErrors.badRequest('Attach a file in the "avatar" field');
```

2 Hard limits: size, number of files and fields.

```
    // Peek at the first bytes, then stream the rest straight to disk (no buffering)
    const chunks = file.file[Symbol.asyncIterator]();  3
    const first = await chunks.next();
    const head = first.done ? Buffer.alloc(0) : (first.value as Buffer);
    const ext = detect(head);
    if (!ext) {
      file.file.resume(); // drain and discard  4
      throw app.httpErrors.unsupportedMediaType('Only PNG or JPEG images');
    }

    // Never use the client's filename on disk
    const name = `${randomUUID()}.${ext}`;  5
    const target = path.join(dir, name);
    await pipeline(async function* () {  6
      yield head;
      for await (const chunk of { [Symbol.asyncIterator]: () => chunks })
        yield chunk;
    }, createWriteStream(target));
```

3	Read only the first chunk to sniff the type, then keep streaming.
4	Rejected uploads are drained so the connection stays usable.
5	A random name: `../../etc/passwd.png` can't escape the directory (tested).
6	Stream to disk: memory use doesn't grow with file size.

```
    if (file.file.truncated) {  7
      // hit the size limit mid-stream
      await unlink(target);
      throw app.httpErrors.payloadTooLarge(`Max ${maxBytes / 1024 / 1024} MB`);
    }
    return reply
      .code(201)
      .send({ url: `/avatars/${name}`, original: file.filename });
  });

  return app;
}
```

7	Size limit hit mid-stream: delete the partial file, answer 413.

What the tests prove (`test/uploads.test.ts`, all passing):

Test	Verified behaviour
a PNG is stored under a random name	201; one file named `.png` in the upload dir.
a script renamed to .png is rejected by its content	415.
a file over the limit is rejected and the partial file removed	413; no file left behind.

19.5 Real-world example: support chat rooms (WebSockets)

The situation. Customers and support agents chat in a room per ticket. Only authenticated users may connect; rooms must not see each other's messages.

Why it works well in Fastify:

- WebSocket routes are normal routes: same hooks, same auth.
- Rejecting in `preValidation` means no socket is ever opened.
- Tested with Node's built-in WebSocket client.

```
// src/chat/app.ts
import Fastify from 'fastify';
import websocket from '@fastify/websocket';
import type { WebSocket } from 'ws';

// Support chat: customers and agents talk in per-ticket rooms
export async function buildChatApp({ tokens }: { tokens: Record<string, string> }) {
  const app = Fastify();
  await app.register(websocket, { options: { maxPayload: 16 * 1024 } });  1

  const rooms = new Map<string, Map<WebSocket, string>>(); // room -> socket -> user  2
  const broadcast = (room: string, message: object) => {
    for (const socket of rooms.get(room)?.keys() ?? [])
      socket.send(JSON.stringify(message));
  };
```

1 Cap message size: a 100 MB frame can't exhaust memory.

2 Room membership in memory; multiple instances need a shared pub/sub.

```
  app.get<{ Params: { room: string }; Querystring: { token?: string } }>(
    '/rooms/:room',
    {
      websocket: true,
      // Normal hooks run on the HTTP upgrade request: reject before the socket opens
      preValidation: async (request, reply) => {  3
        if (!tokens[request.query.token ?? '']) {
          return reply.code(401).send({ message: 'Invalid token' });
        }
      },
    },
    (socket, request) => {
      const { room } = request.params;
      const user = tokens[request.query.token!];
      if (!rooms.has(room)) rooms.set(room, new Map());
      rooms.get(room)!.set(socket, user);
      broadcast(room, { type: 'joined', user, online: rooms.get(room)!.size });
```

3 Regular hooks run on the HTTP upgrade request: authenticate **before** the socket opens.

```
      socket.on('message', (raw) => {
        const text = String(raw).trim().slice(0, 2000);  4
        if (text) broadcast(room, { type: 'message', user, text, at: Date.now() });
      });
      socket.on('close', () => {  5
        rooms.get(room)!.delete(socket);
        broadcast(room, { type: 'left', user, online: rooms.get(room)!.size });
      });
    },
  );

  return app;
}
```

4 Never trust message content: trim and bound it.

5 Clean up membership and tell the room.

What the tests prove (`test/chat.test.ts`, all passing):

Test	Verified behaviour
customer and agent in one ticket room see each other's messages	Join notice, message delivered with the sender, leave notice with online=1.
rooms are isolated	A message in ticket-1 never reaches ticket-2.
an invalid token is refused before the socket opens	The client's connection fails.

Part IV — TypeScript, Testing & Production

20. TypeScript in Depth

20.1 Route generics (no type provider)

```
interface GetUser {
  Params: { id: string };
  Querystring: { expand?: 'posts' | 'comments' };
  Headers: { 'x-tenant-id': string };
  Reply: { 200: { id: string; name: string }; 404: { message: string } };
}

app.get<GetUser>('/users/:id', async (request, reply) => {
  request.params.id;                // string
  request.query.expand;             // 'posts' | 'comments' | undefined
  if (!found) return reply.code(404).send({ message: 'nope' });
  return { id: request.params.id, name: 'Ann' };
});
```

Generics give types but **no runtime validation** — you still need a schema. Type providers (Chapter 7.5) give both from a single definition, so prefer them.

20.2 Typed plugins

```
import type { FastifyPluginAsync } from 'fastify';
import type { FastifyPluginAsyncTypebox } from '@fastify/type-provider-typebox';

// Plain plugin with typed options
const plugin: FastifyPluginAsync<{ prefixMsg: string }> = async (app, opts) => { };

// Plugin whose routes get TypeBox inference automatically
const routes: FastifyPluginAsyncTypebox = async (app) => {
  app.get('/x', { schema: { querystring: Type.Object({ n: Type.Integer() }) } },
    async (req) => req.query.n * 2);      // req.query.n is number
};
```

20.3 Useful types

Type	Use
`FastifyInstance`	The app / plugin instance (augment for decorators).
`FastifyRequest`, `FastifyReply`	Handler/hook parameters (augment for request/reply decorators).
`FastifyPluginAsync<Opts>`	Async plugin with typed options.
`FastifyServerOptions`	Options passed to `Fastify()` (for `buildApp(opts)`).
`RouteShorthandOptions`	Options object of `app.get(url, opts, handler)`.
`FastifyError`	Error type in error handlers (has statusCode, code, validation).

21. Testing

21.1 Separate app construction from listening

```
// src/app.ts - builds the app, never listens
export async function buildApp(opts: FastifyServerOptions = {}) {
  const app = Fastify(opts);
  await app.register(/* plugins, routes */);
  return app;
}

// src/server.ts - production entry point
const app = await buildApp({ logger: true });
await app.listen({ port: 3000, host: '0.0.0.0' });
```

21.2 inject(): HTTP tests without a network

```
// test/users.test.ts  - run with: node --import tsx --test test/*.test.ts
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { buildApp } from '../src/app';

test('GET /users/:id returns the user', async (t) => {
  const app = await buildApp();
  t.after(() => app.close());

  const res = await app.inject({ method: 'GET', url: '/users/1' });

  assert.equal(res.statusCode, 200);
  assert.deepEqual(res.json(), { id: 1, name: 'Ann' });
  assert.match(res.headers['content-type'] as string, /application\/json/);
});

test('POST /users validates body', async (t) => {
  const app = await buildApp();
  t.after(() => app.close());

  const res = await app.inject({
    method: 'POST', url: '/users',
    headers: { 'x-tenant-id': 't1' },
    payload: { email: 'not-an-email', name: 'A' },
  });
  assert.equal(res.statusCode, 400);
});
```

21.3 Testing plugins and mocking dependencies

```
test('uses a fake repository', async (t) => {
  const app = Fastify();
  app.decorate('repo', { find: async () => ({ id: 1, name: 'Fake' }) });  // mock
  await app.register(usersRoutes, { prefix: '/users' });
  t.after(() => app.close());

  const res = await app.inject('/users/1');
  assert.equal(res.json().name, 'Fake');
});

// Chainable API
const res = await app.inject().get('/users').headers({ authorization: 'Bearer x' }).end();
```

Tip: Tests run in milliseconds because `inject()` calls the router directly (via light-my-request) — no ports, no sockets. Jest, Vitest and tap work the same way.

21.4 The real-world examples test suite

All 14 scenarios live in `fastify-examples/`, one test file each, using Node's test runner. Most use `inject()`; streaming, uploads, WebSockets and shutdown listen on a random port and use `fetch` / `WebSocket`.

```
npm test        # node --import tsx --test --test-concurrency=4 test/*.test.ts
# tests 54, pass 54, fail 0   (about 10 s on a laptop)
```

Test file	Tests	Technique worth copying
`catalog`	5	Assert router 404s vs handler 404s
`downloads`	4	Async generator as a fake DB cursor
`order-tracking`	2	Read SSE with fetch; abort and check listeners
`signup`	5	Inspect `app.swagger()` in a test
`multi-tenant`	4	Host header selects the tenant
`observability`	4	Capture logs via a custom stream
`problem-details`	5	Assert the full error contract
`webhooks`	5	Sign like the provider; fixed clock
`inventory`	4	In-memory SQLite; verify rollback
`auth-rbac`	4	Replay a refresh token like an attacker
`security`	5	X-Forwarded-For to simulate many clients
`uploads`	3	Real multipart via FormData + Blob
`chat`	3	Built-in WebSocket client
`graceful-shutdown`	1	Close during an in-flight request

22. Project Structure & Autoload

```
src/
  app.ts              # buildApp(): registers plugins then routes
  server.ts           # listen + graceful shutdown
  plugins/            # fp-wrapped, shared (env, db, auth, sensible, swagger)
    env.ts
    db.ts
    auth.ts
  routes/             # encapsulated feature plugins; folder name = URL prefix
    health/index.ts   # -> /health
    users/index.ts    # -> /users
    users/_id/posts.ts
  schemas/            # TypeBox / JSON schemas
  services/           # business logic, independent of Fastify
test/
```

@fastify/autoload registers every file in a folder

```
import AutoLoad from '@fastify/autoload';
import path from 'node:path';

export async function buildApp(opts = {}) {
  const app = Fastify(opts);
  await app.register(AutoLoad, { dir: path.join(__dirname, 'plugins') });
  await app.register(AutoLoad, {
    dir: path.join(__dirname, 'routes'),
    options: { prefix: '/api' },       // /api/users, /api/health ...
  });
  return app;
}
```

Autoload keeps `app.ts` tiny as the app grows. Explicit `register()` calls (as in the Part V project) are easier to follow for small apps.

23. Performance

- **Define response schemas** — the biggest single win (fast-json-stringify).
- Keep handlers `async` and never block the event loop (no sync crypto/fs, no giant JSON parsing in handlers).
- Log at `info` or higher in production; avoid pino-pretty in production.
- Reuse connections (DB pools, HTTP agents with keep-alive, `undici`).
- Cache with `@fastify/caching` / Redis; compress with `@fastify/compress` (or at the proxy).
- Scale by running several processes/containers behind a load balancer.
- Protect under load with `@fastify/under-pressure` (returns 503 when the event loop is saturated).

```
npx autocannon -c 100 -d 10 http://localhost:3000/     # benchmark
npx clinic doctor -- node dist/server.js               # diagnose bottlenecks
```

```
import underPressure from '@fastify/under-pressure';
await app.register(underPressure, {
  maxEventLoopDelay: 1000,
  maxHeapUsedBytes: 1_000_000_000,
  exposeStatusRoute: '/health/live',
});
```

24. Deployment

24.1 Graceful shutdown

```
npm install close-with-grace
```

```
import closeWithGrace from 'close-with-grace';

const app = await buildApp({ logger: true });

closeWithGrace({ delay: 10_000 }, async ({ signal, err }) => {
  if (err) app.log.error({ err }, 'closing because of error');
  else app.log.info({ signal }, 'shutting down');
  await app.close();       // stops accepting, finishes in-flight, runs onClose hooks
});

await app.listen({ port: app.config.PORT, host: '0.0.0.0' });
```

24.2 Dockerfile

```
FROM node:22-slim AS build
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build && npm prune --omit=dev

FROM node:22-slim
WORKDIR /app
ENV NODE_ENV=production
COPY --from=build /app/node_modules ./node_modules
COPY --from=build /app/dist ./dist
COPY package.json ./
USER node
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

24.3 Production checklist

Item	How
Listen on all interfaces	`host: '0.0.0.0'`
Behind a proxy / LB	`trustProxy: true` (or the proxy's IP list)
Health checks	`/health` route + under-pressure for liveness/readiness
Timeouts	`requestTimeout`, `connectionTimeout`, keep-alive shorter than the LB's
Logs	JSON to stdout, redact secrets, ship to your log platform
Config	`@fastify/env` with a strict schema; secrets from the environment
Shutdown	close-with-grace; `onClose` hooks close resources
Security	helmet, cors, rate-limit, response schemas, small bodyLimit

24.4 Real-world example: zero-downtime deploys (graceful shutdown)

The situation. Kubernetes sends SIGTERM during every rolling deploy. Customers downloading a report at that moment must get the full response; the database and metrics must be flushed after the last request.

Why it works well in Fastify:

- Lifecycle hooks: `preClose` (stop advertising readiness), `onClose` (release resources).
- `app.close()` waits for in-flight requests.
- The test measured a 71-second shutdown before the keep-alive fix.

```ts
// src/graceful-shutdown/app.ts
import Fastify from 'fastify';
import { setTimeout as delay } from 'node:timers/promises';

export interface Resources {
  closeDb(): Promise<void>;
  flushMetrics(): Promise<void>;
}

// What a rolling deploy needs: finish in-flight work, release resources, exit
export async function buildShutdownApp(resources: Resources) {
  const app = Fastify({
    forceCloseConnections: 'idle', // drop idle keep-alive sockets on close  1
    requestTimeout: 30_000, // bound how long a request may hold shutdown
  });
```

1 Idle keep-alive sockets are closed when shutdown starts...

```ts
  let shuttingDown = false;

  // Readiness probe: fails as soon as shutdown starts, so the load balancer stops
  // sending new traffic while in-flight requests finish
  app.get('/ready', async (_request, reply) =>  2
    shuttingDown ? reply.code(503).send({ ready: false }) : { ready: true },
  );

  app.get('/reports/monthly', async () => {
    await delay(500); // a slow request that must not be cut off
    return { report: 'september', rows: 12_840 };
  });

  app.addHook('preClose', async () => {  3
    shuttingDown = true;
  });
  // Without this, a keep-alive socket that becomes idle AFTER close() started keeps
  // the server open until keepAliveTimeout (72 s by default, measured: 71 s)
  app.addHook('onSend', async (_request, reply, payload) => {  4
    if (shuttingDown) reply.header('connection', 'close');
    return payload;
  });
  app.addHook('onClose', async () => {  5
    // after the server stopped accepting
    await resources.flushMetrics();
    await resources.closeDb();
  });
```

2 Readiness probe flips to 503 so the load balancer stops routing here.

3 `preClose` runs first, while the server still serves in-flight requests.

4 ...but sockets that become idle *later* are not. Found by the test: close took 71 s until responses sent `Connection: close`.

5 Flush and close resources only after the last request finished.

```ts
  return app;
}

// server.ts in production:
//   closeWithGrace({ delay: 10_000 }, async () => { await app.close(); });
```

```ts
// test/graceful-shutdown.test.ts
import { test } from 'node:test';
import assert from 'node:assert/strict';
import { setTimeout as delay } from 'node:timers/promises';
import { buildShutdownApp } from '../src/graceful-shutdown/app';

test('a deploy finishes in-flight requests, then releases resources', async () => {
  const events: string[] = [];
  const app = await buildShutdownApp({
    closeDb: async () => {
      events.push('db closed');
    },
    flushMetrics: async () => {
      events.push('metrics flushed');
    },
  });
  const base = await app.listen({ port: 0, host: '127.0.0.1' });
  assert.equal((await fetch(`${base}/ready`)).status, 200);

  // A customer is downloading a report when SIGTERM arrives
  const inFlight = fetch(`${base}/reports/monthly`).then(async (r) => {  // 1
    events.push(`report ${r.status}`);
    return r.json();
  });
  await delay(100);
  const closing = app.close().then(() => events.push('closed'));  // 2

  const report = await inFlight; // complete response, not cut off
  assert.equal(report.rows, 12_840);  // 3
  await closing;
  assert.ok(events.includes('report 200'));
  assert.deepEqual(
    events.filter((e) => e !== 'report 200'),
    ['metrics flushed', 'db closed', 'closed'], // resources released, then done
  );

  // After close, the port refuses new connections
  await assert.rejects(fetch(`${base}/ready`));  // 4
});
```

1	A slow request is in flight when the deploy starts.
2	What `close-with-grace` does on SIGTERM.
3	The in-flight request completed fully.
4	New connections are refused after close.

Watch out: Measured: after `close()` started, the in-flight request's keep-alive socket became idle and held the server open for **71 seconds** (Fastify's default `keepAliveTimeout` is 72 s). Sending `Connection: close` during shutdown fixed it (< 1 s).

Part V — End-to-End Project: Tasks API

25. Building a Production-Style REST API

A complete, runnable service combining the concepts: TypeBox schemas with full type inference, environment validation, JWT authentication, per-user authorization, security plugins, Swagger docs, a central error handler, graceful shutdown and tests. Storage is an in-memory repository plugin that you can later swap for a database plugin without touching routes.

Method & URL	Auth	Description
`GET /health`	—	Liveness check
`POST /auth/login`	—	Returns a JWT (rate limited: 5/min)
`GET /tasks?done=&limit=&offset=`	JWT	List own tasks with filters
`POST /tasks`	JWT	Create a task
`GET /tasks/:id`	JWT	Get one task (404 if not owner)
`PATCH /tasks/:id`	JWT	Update title/done
`DELETE /tasks/:id`	JWT	Delete (204)
`GET /docs`	—	Swagger UI

25.1 Install

```
mkdir tasks-api && cd tasks-api && npm init -y
npm install fastify fastify-plugin @fastify/sensible @fastify/env @fastify/jwt \
            @fastify/helmet @fastify/cors @fastify/rate-limit @fastify/swagger \
            @fastify/swagger-ui @fastify/type-provider-typebox close-with-grace
npm install -D typescript tsx @types/node pino-pretty
```

.env (add to .gitignore)

```
PORT=3000
HOST=0.0.0.0
LOG_LEVEL=info
JWT_SECRET=replace-with-a-long-random-string-at-least-32-chars
```

Layout (tsconfig.json and scripts as in Chapter 3.3)

```
tasks-api/
  src/
    app.ts  server.ts
    schemas/task.ts
    plugins/env.ts  security.ts  swagger.ts  store.ts  auth.ts
    routes/health.ts  auth.ts  tasks.ts
  test/tasks.test.ts
  .env  package.json  tsconfig.json
```

25.2 Schemas (single source of truth for validation, types and docs)

```
// src/schemas/task.ts
import { Type, Static } from '@fastify/type-provider-typebox';

export const Task = Type.Object({  1
  id: Type.String(),
  title: Type.String(),
  done: Type.Boolean(),
  ownerId: Type.String(),
  createdAt: Type.String({ format: 'date-time' }),
});
export type Task = Static<typeof Task>;  2

const Title = Type.String({ minLength: 1, maxLength: 200 });

export const CreateTask = Type.Object({ title: Title }, { additionalProperties: false });  3
export const UpdateTask = Type.Object(
  { title: Type.Optional(Title), done: Type.Optional(Type.Boolean()) },
  { additionalProperties: false, minProperties: 1 },  4
);
export const TaskParams = Type.Object({ id: Type.String() });
export const ListQuery = Type.Object({
  done: Type.Optional(Type.Boolean()),
  limit: Type.Integer({ minimum: 1, maximum: 100, default: 20 }),  5
  offset: Type.Integer({ minimum: 0, default: 0 }),
});
export type ListQuery = Static<typeof ListQuery>;

export const ErrorReply = Type.Object({
  statusCode: Type.Integer(), error: Type.String(), message: Type.String(),
});
```

1	One TypeBox schema drives validation, response serialization, Swagger docs **and** TypeScript types.
2	`Static` derives the TypeScript type from the schema — no duplicate interface to keep in sync.
3	Unknown fields are stripped (Ajv `removeAdditional`), so clients can't set `ownerId` or `done` on create.
4	`minProperties: 1` rejects an empty PATCH body with a 400.
5	Query strings are coerced (`"10"` → 10), defaulted and bounded before your handler runs.

25.3 Plugins - environment

```
// src/plugins/env.ts
import fp from 'fastify-plugin';
import fastifyEnv from '@fastify/env';

const schema = {
  type: 'object',
  required: ['JWT_SECRET'],
  properties: {
    PORT: { type: 'number', default: 3000 },
    HOST: { type: 'string', default: '0.0.0.0' },
    JWT_SECRET: { type: 'string', minLength: 32 },
  },
};

declare module 'fastify' {
  interface FastifyInstance {
    config: { PORT: number; HOST: string; JWT_SECRET: string };
  }
}

export default fp(async (app) => {
  await app.register(fastifyEnv, { schema, dotenv: true });
}, { name: 'env' });
```

25.4 Plugins - security and docs

```
// src/plugins/security.ts
import fp from 'fastify-plugin';
import helmet from '@fastify/helmet';
import cors from '@fastify/cors';
import rateLimit from '@fastify/rate-limit';

export default fp(async (app) => {
  await app.register(helmet, { contentSecurityPolicy: false });  // CSP off for /docs
  await app.register(cors, { origin: true });                    // tighten in prod
  await app.register(rateLimit, { max: 100, timeWindow: '1 minute' });
}, { name: 'security' });
```

```
// src/plugins/swagger.ts
import fp from 'fastify-plugin';
import swagger from '@fastify/swagger';
import swaggerUi from '@fastify/swagger-ui';

export default fp(async (app) => {
  await app.register(swagger, {
    openapi: {
      info: { title: 'Tasks API', version: '1.0.0' },
      components: { securitySchemes: { bearerAuth: { type: 'http', scheme: 'bearer' } } },
    },
  });
  await app.register(swaggerUi, { routePrefix: '/docs' });
}, { name: 'swagger' });
```

25.5 Plugins - repository (swap for Postgres/Prisma later)

```
// src/plugins/store.ts
import fp from 'fastify-plugin';
import { randomUUID, scryptSync, timingSafeEqual } from 'node:crypto';
import type { Task, ListQuery } from '../schemas/task';

interface User { id: string; username: string; salt: string; hash: Buffer }
const hashPw = (pw: string, salt: string) => scryptSync(pw, salt, 32);  1

export class Store {
  private tasks = new Map<string, Task>();
  private users = new Map<string, User>();

  constructor() {                                // demo users - use a real DB!
    for (const username of ['alice', 'bob']) {
      const salt = randomUUID();
      this.users.set(username,
        { id: randomUUID(), username, salt, hash: hashPw('password123', salt) });
    }
  }
```

1 Passwords are stored as salted scrypt hashes, never in plain text.

```
  verifyUser(username: string, password: string): User | undefined {
    const u = this.users.get(username);
    if (!u) return undefined;
    return timingSafeEqual(u.hash, hashPw(password, u.salt)) ? u : undefined;  2
  }

  listTasks(ownerId: string, q: ListQuery): Task[] {
    return [...this.tasks.values()]
      .filter((t) => t.ownerId === ownerId && (q.done === undefined || t.done === q.done))
      .slice(q.offset, q.offset + q.limit);
  }

  getTask(id: string) { return this.tasks.get(id); }

  createTask(ownerId: string, title: string): Task {
    const task: Task = { id: randomUUID(), title, done: false, ownerId,
                         createdAt: new Date().toISOString() };
    this.tasks.set(task.id, task);
    return task;
  }
```

2 `timingSafeEqual` prevents timing attacks when comparing hashes.

```
  updateTask(id: string, patch: Partial<Pick<Task, 'title' | 'done'>>): Task {
    const task = { ...this.tasks.get(id)!, ...patch };
    this.tasks.set(id, task);
    return task;
  }

  deleteTask(id: string) { this.tasks.delete(id); }
}

declare module 'fastify' { interface FastifyInstance { store: Store } }  3

export default fp(async (app) => {
  app.decorate('store', new Store());  4
}, { name: 'store' });
```

3 Declaration merging types `app.store` everywhere.

4 Wrapped in `fp()`, so the decorator is shared app-wide. Swap this plugin for a database-backed one without touching routes.

25.6 Plugins - authentication

```
// src/plugins/auth.ts
import fp from 'fastify-plugin';
import jwt from '@fastify/jwt';
import type { FastifyRequest } from 'fastify';

declare module '@fastify/jwt' {  1
  interface FastifyJWT {
    payload: { sub: string; username: string };
    user: { sub: string; username: string };
  }
}
declare module 'fastify' {
  interface FastifyInstance { authenticate(request: FastifyRequest): Promise<void> }
}
```

1 Types what you sign (`payload`) and what `request.user` holds after verification.

```
export default fp(async (app) => {
  await app.register(jwt, { secret: app.config.JWT_SECRET, sign: { expiresIn: '1h' } });  2

  app.decorate('authenticate', async (request: FastifyRequest) => {
    try {
      await request.jwtVerify();  3
    } catch {
      throw app.httpErrors.unauthorized('Invalid or missing token');  4
    }
  });
}, { name: 'auth', dependencies: ['env'] });  5
```

2	Secret comes from validated env config; tokens expire after one hour.
3	Checks signature and expiry, then sets `request.user`.
4	Any failure becomes a clean 401 via `@fastify/sensible`.
5	Plugin metadata: Fastify verifies that `env` was registered first.

25.7 Routes - health and login

```
// src/routes/health.ts
import type { FastifyPluginAsync } from 'fastify';

const health: FastifyPluginAsync = async (app) => {
  app.get('/health', {
    schema: { tags: ['system'],
              response: { 200: { type: 'object',
                                 properties: { status: { type: 'string' },
                                               uptime: { type: 'number' } } } } } },
  }, async () => ({ status: 'ok', uptime: process.uptime() }));
};
export default health;
```

```
// src/routes/auth.ts
import { Type, FastifyPluginAsyncTypebox } from '@fastify/type-provider-typebox';
import { ErrorReply } from '../schemas/task';

const authRoutes: FastifyPluginAsyncTypebox = async (app) => {
  app.post('/login', {
    config: { rateLimit: { max: 5, timeWindow: '1 minute' } },   // brute-force guard  1
    schema: {
      tags: ['auth'],
      body: Type.Object({ username: Type.String(), password: Type.String() }),
      response: { 200: Type.Object({ token: Type.String() }), 401: ErrorReply },
    },
  }, async (request) => {
    const { username, password } = request.body;          // fully typed  2
    const user = app.store.verifyUser(username, password);
    if (!user) throw app.httpErrors.unauthorized('Invalid credentials');
    return { token: app.jwt.sign({ sub: user.id, username: user.username }) };  3
  });
};
export default authRoutes;
```

1	Per-route rate limit against brute force: the 6th attempt in a minute gets 429.
2	`request.body` is typed from the schema by the TypeBox type provider.
3	Sign only what you need; `sub` identifies the user on later requests.

25.8 Routes - tasks CRUD

```
// src/routes/tasks.ts
import { Type, FastifyPluginAsyncTypebox } from '@fastify/type-provider-typebox';
import type { FastifyRequest } from 'fastify';
import { Task, CreateTask, UpdateTask, TaskParams, ListQuery, ErrorReply }
  from '../schemas/task';

const tasksRoutes: FastifyPluginAsyncTypebox = async (app) => {
  // Every route in this plugin requires a valid JWT (encapsulated hook)
  app.addHook('onRequest', app.authenticate);  1

  const common = { tags: ['tasks'], security: [{ bearerAuth: [] }] };  2

  const getOwned = (request: FastifyRequest, id: string) => {
    const task = app.store.getTask(id);
    if (!task || task.ownerId !== request.user.sub) {  3
      throw app.httpErrors.notFound('Task not found');   // don't reveal existence
    }
    return task;
  };
```

1. Encapsulated hook: protects every route in **this** plugin only; `/health` and `/auth` stay public.
2. Shared Swagger metadata: groups the routes and shows the lock icon in `/docs`.
3. Authorization: another user's task returns **404, not 403**, so IDs can't be probed.

```
  app.get('/', {
    schema: { ...common, querystring: ListQuery, response: { 200: Type.Array(Task) } },
  }, async (request) => app.store.listTasks(request.user.sub, request.query));  4

  app.post('/', {
    schema: { ...common, body: CreateTask, response: { 201: Task } },
  }, async (request, reply) => {
    reply.code(201);  5
    return app.store.createTask(request.user.sub, request.body.title);
  });

  app.get('/:id', {
    schema: { ...common, params: TaskParams, response: { 200: Task, 404: ErrorReply } },
  }, async (request) => getOwned(request, request.params.id));
```

4. `request.query` is already typed, coerced and defaulted.
5. Set the status, then just return the object; the 201 response schema serializes it.

```
  app.patch('/:id', {
    schema: { ...common, params: TaskParams, body: UpdateTask,
              response: { 200: Task, 404: ErrorReply } },
  }, async (request) => {
    getOwned(request, request.params.id);
    return app.store.updateTask(request.params.id, request.body);
  });

  app.delete('/:id', {
    schema: { ...common, params: TaskParams, response: { 204: Type.Null(),
                                                         404: ErrorReply } },
  }, async (request, reply) => {
    getOwned(request, request.params.id);
    app.store.deleteTask(request.params.id);
    return reply.code(204).send(null);  6
  });
};
export default tasksRoutes;
```

6. With a `Type.Null()` 204 schema, `send` needs an explicit `null`.

25.9 App builder

```
// src/app.ts
import Fastify, { FastifyServerOptions, FastifyError } from 'fastify';
import { STATUS_CODES } from 'node:http';
import sensible from '@fastify/sensible';
import type { TypeBoxTypeProvider } from '@fastify/type-provider-typebox';
import env from './plugins/env';
import security from './plugins/security';
import swagger from './plugins/swagger';
import store from './plugins/store';
import auth from './plugins/auth';
import health from './routes/health';
import authRoutes from './routes/auth';
import tasksRoutes from './routes/tasks';

export async function buildApp(opts: FastifyServerOptions = {}) {
  const app = Fastify(opts).withTypeProvider<TypeBoxTypeProvider>();  1
```

1 Enables schema-to-type inference for every route registered on this instance.

```
  // 1. Shared plugins (order matters: env first)
  await app.register(env);  2
  await app.register(sensible);
  await app.register(security);
  await app.register(swagger);            // before routes  3
  await app.register(store);
  await app.register(auth);

  // 2. Central error handling
  app.setErrorHandler((error: FastifyError, request, reply) => {  4
    const status = error.statusCode ?? 500;
    if (status >= 500) request.log.error({ err: error }, 'unhandled error');
    reply.code(status).send({
      statusCode: status,
      error: STATUS_CODES[status] ?? 'Error',
      message: status >= 500 ? 'Something went wrong' : error.message,
    });
  });
```

2 Order matters: config first, because later plugins read `app.config`.

3 Swagger must be registered before the routes it documents.

4 One place for error responses: consistent JSON, HTTP status text via `STATUS_CODES`, and 5xx details hidden from clients.

```
  // 3. Feature routes (encapsulated)
  await app.register(health);
  await app.register(authRoutes, { prefix: '/auth' });
  await app.register(tasksRoutes, { prefix: '/tasks' });  5

  return app;
}
```

5 Feature plugins get a URL prefix and their own encapsulated hooks.

25.10 Server entry point

```
// src/server.ts
import closeWithGrace from 'close-with-grace';
import { buildApp } from './app';

async function main() {
  const app = await buildApp({
    logger: {
      level: process.env.LOG_LEVEL ?? 'info',
      redact: ['req.headers.authorization'],  1
      transport: process.env.NODE_ENV === 'production'
        ? undefined : { target: 'pino-pretty' },
    },
    trustProxy: true,  2
    requestTimeout: 30_000,
  });
```

1. Bearer tokens never reach the logs.
2. Behind a load balancer: `request.ip` and rate limits use the real client IP.

```
  closeWithGrace({ delay: 10_000 }, async ({ signal, err }) => {  3
    if (err) app.log.error({ err }, 'shutting down after error');
    else app.log.info({ signal }, 'shutting down');
    await app.close();
  });

  await app.listen({ port: app.config.PORT, host: app.config.HOST });  4
}

main().catch((err) => { console.error(err); process.exit(1); });
```

3. Graceful shutdown on SIGINT/SIGTERM or crash: stop accepting, finish in-flight requests, run `onClose` hooks.
4. `HOST` defaults to `0.0.0.0` so the server is reachable inside containers.

25.11 Tests

```
// test/tasks.test.ts
import { test, before, after } from 'node:test';
import assert from 'node:assert/strict';
import { buildApp } from '../src/app';

process.env.JWT_SECRET = 'test-secret-test-secret-test-secret-123';  1
let app: Awaited<ReturnType<typeof buildApp>>;
let token: string;

before(async () => {
  app = await buildApp();
  const res = await app.inject({ method: 'POST', url: '/auth/login',  2
    payload: { username: 'alice', password: 'password123' } });
  token = res.json().token;
});
after(() => app.close());
```

1. Set required config before building the app; env validation runs at boot.
2. `inject()` runs the full lifecycle (hooks, validation, serialization) with no network.

```
const auth = () => ({ authorization: `Bearer ${token}` });

test('rejects requests without a token', async () => {
  const res = await app.inject({ method: 'GET', url: '/tasks' });
  assert.equal(res.statusCode, 401);
});

test('creates, lists, updates and deletes a task', async () => {
  const created = await app.inject({ method: 'POST', url: '/tasks', headers: auth(),
                                     payload: { title: 'Write tests' } });
  assert.equal(created.statusCode, 201);
  const { id } = created.json();

  const list = await app.inject({ url: '/tasks?done=false', headers: auth() });
  assert.equal(list.json().length, 1);
```

```
  const patched = await app.inject({ method: 'PATCH', url: `/tasks/${id}`,
                                     headers: auth(), payload: { done: true } });
  assert.equal(patched.json().done, true);

  const del = await app.inject({ method: 'DELETE', url: `/tasks/${id}`,
                                 headers: auth() });
  assert.equal(del.statusCode, 204);
});

test('validates input', async () => {  3
  const res = await app.inject({ method: 'POST', url: '/tasks', headers: auth(),
                                 payload: { title: '' } });
  assert.equal(res.statusCode, 400);
});
```

3 Schema rules are part of the contract — test them like business logic.

25.12 Run it

```
npm run dev          # tsx watch src/server.ts
npm test             # node --import tsx --test test/*.test.ts
```

On Windows PowerShell use `curl.exe` instead of the `curl` alias.

```
# 1. Log in
curl -X POST localhost:3000/auth/login -H "Content-Type: application/json" \
     -d '{"username":"alice","password":"password123"}'
# {"token":"eyJhbGciOi..."}
TOKEN=eyJhbGciOi...

# 2. Create
curl -X POST localhost:3000/tasks -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" -d '{"title":"Learn Fastify"}'
# 201 {"id":"3f2c...","title":"Learn Fastify","done":false,
#      "ownerId":"...","createdAt":"..."}

# 3. List with coerced query params
curl "localhost:3000/tasks?done=false&limit=10" -H "Authorization: Bearer $TOKEN"

# 4. Update / delete
curl -X PATCH localhost:3000/tasks/3f2c... -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" -d '{"done":true}'
curl -X DELETE localhost:3000/tasks/3f2c... -H "Authorization: Bearer $TOKEN"   # 204

# 5. Errors
curl localhost:3000/tasks                        # 401 Invalid or missing token
curl -X POST localhost:3000/tasks -H "Authorization: Bearer $TOKEN" \
     -H "Content-Type: application/json" -d '{"title":"","x":1}'   # 400 validation

# 6. Docs
open http://localhost:3000/docs
```

25.13 What to notice

- **No manual validation code**: TypeBox schemas validate, coerce (`done=false` → boolean), set defaults (`limit=20`) and type `request.body`/`request.query`.
- **Encapsulated auth**: the `onRequest` hook in `tasks.ts` protects only task routes; `/health` and `/auth` stay public.
- **Response schemas** generate Swagger docs and guarantee output shape.
- **Swappable storage**: replace `plugins/store.ts` with a Postgres/Prisma plugin exposing the same methods; routes and tests stay unchanged.
- **Fast tests**: `inject()` exercises the full lifecycle (hooks, validation, serialization) without opening a port.
- **Unknown fields are stripped, not rejected**: `{"title":"ok","x":1}` creates a task (Ajv `removeAdditional`). Set Ajv `removeAdditional: false` if you prefer a 400.
- **Rate limit is real**: the 6th login attempt within a minute returns `429 Too Many Requests`; every response carries `x-ratelimit-*` headers.

25.14 Next steps

- Persist data with `@fastify/postgres` or Prisma (Chapter 15).
- Add refresh tokens and roles (`preHandler` authorization).
- Add `@fastify/under-pressure` and containerize (Chapters 23-24).
- Split into `@fastify/autoload` folders as it grows (Chapter 22).

Appendices

A. Express to Fastify Cheat Sheet

Express	Fastify
`const app = express()`	`const app = Fastify({ logger: true })`
`app.use(express.json())`	Built in (JSON parser)
`app.use(middleware)`	`app.addHook('onRequest', fn)` or a plugin
`express.Router()`	`app.register(plugin, { prefix })`
`req.params / req.query / req.body`	`request.params / query / body`
`res.status(201).json(x)`	`reply.code(201); return x`
`res.set('X', 'y')`	`reply.header('X', 'y')`
`res.redirect(url)`	`reply.redirect(url)`
`res.sendFile()`	`@fastify/static` + `reply.sendFile()`
`next(err)`	`throw err`
`app.use((err, req, res, next) => ...)`	`app.setErrorHandler(fn)`
`req.user = ...` in middleware	`decorateRequest('user', null)` + hook
`cors()`, `helmet()`	`@fastify/cors`, `@fastify/helmet`
`supertest(app)`	`app.inject()`
Existing Express middleware	`@fastify/express` or `@fastify/middie` (migration bridge)

B. Official Plugins Reference

Plugin	Purpose
`@fastify/autoload`	Load plugins/routes from folders
`@fastify/sensible`	HTTP errors, assert, useful utilities
`@fastify/env`	Validated environment configuration
`@fastify/cors`, `@fastify/helmet`	CORS and security headers
`@fastify/rate-limit`	Rate limiting (memory or Redis)
`@fastify/jwt`, `@fastify/auth`	JWT and auth strategy composition
`@fastify/cookie`, `@fastify/session`	Cookies and sessions
`@fastify/oauth2`	OAuth2 login (Google, GitHub, ...)
`@fastify/swagger`, `@fastify/swagger-ui`	OpenAPI docs
`@fastify/static`, `@fastify/view`	Static files, template rendering
`@fastify/multipart`, `@fastify/formbody`	Uploads and form bodies
`@fastify/websocket`	WebSocket routes

Plugin	Purpose
`@fastify/compress`, `@fastify/caching`	Compression and caching headers
`@fastify/postgres`, `@fastify/mongodb`, `@fastify/redis`	Database connectors
`@fastify/under-pressure`	Load shedding and health status
`@fastify/http-proxy`	Reverse proxy / API gateway
`@fastify/schedule`	Run scheduled jobs (toad-scheduler)

C. Troubleshooting

Symptom	Cause / fix
Server unreachable in Docker	Listening on localhost. Use `host: '0.0.0.0'`.
`FST_ERR_CTP_INVALID_MEDIA_TYPE` / 415	No parser for that content-type. Register `@fastify/formbody`, `@fastify/multipart` or add one.
`FST_ERR_CTP_BODY_TOO_LARGE` / 413	Increase `bodyLimit` (globally or per route).
Decorator undefined in another plugin	Plugin not wrapped with `fastify-plugin`, or registered after the consumer.
Hook doesn't run for some routes	Encapsulation: hook added in a sibling context. Move it to a parent or an fp plugin.
"Reply was already sent"	Both `reply.send()` and a return value in an async handler. Use one.
Fields missing from response	Not listed in the response schema (by design).
Querystring schema rejected at startup	v5 needs full JSON Schema (`type: 'object', properties`).
`FST_ERR_DEC_ALREADY_PRESENT`	Same decorator registered twice (plugin registered twice?). Check with `app.hasDecorator()`.
`TS5108 moduleResolution=node10 removed`	TypeScript 7: set module and moduleResolution to `nodenext`.
`ts-node` / `ts-jest` fail with TS 7	No JS compiler API in TS 7. Use `tsx` and `node --import tsx --test`.
`reply.send()` type error with 204 schema	A `Type.Null()` response expects `reply.code(204).send(null)`.
Rate limit keyed on the body blocks everyone on an IP	The default `onRequest` hook runs before body parsing; set `hook: 'preHandler'` in the route's rateLimit config (17.1).
`app.close()` takes ~72 s	Keep-alive sockets that go idle after close started wait for `keepAliveTimeout`. Send `Connection: close` while shutting down (24.4).
SSE subscribers linger after the tab closes	Disconnects surface on the next write; keep the ping interval short (6.5).
OpenAPI marks a defaulted field as required	Wrap it: `Type.Optional(Type.Boolean({ default: false }))` (7.6).
`FST_ERR_PLUGIN_TIMEOUT`	A plugin never resolved/called done. Await everything; don't mix callback and async styles.

Further resources

- Docs: fastify.dev/docs/latest
- Ecosystem list: fastify.dev/ecosystem
- Examples & demo: github.com/fastify/demo
- Migration guide v4 → v5: fastify.dev/docs/latest/Guides/Migration-Guide-V5
- Community: Fastify Discord and GitHub discussions